

Documentación Técnica

Sistema Contable Computarizado (SCO)

Módulo 8: Análisis Predictivo con Inteligencia Artificial

Versión del Documento: 1.0.0

Fecha de Elaboración: 18 de Junio de 2026

Autores: Equipo de Arquitectura IA y Desarrollo Java

Institución / Proyecto: Proyecto SCO Unificado

2. Tabla de Contenidos

1. Portada
2. Tabla de Contenidos
3. Historial de Versiones
4. Introducción
5. Descripción General del Sistema
6. Arquitectura del Sistema
7. Modelo de Datos
8. Diseño Detallado de Clases
9. Documentación del Motor de IA
10. Interfaces de Usuario
11. Configuración e Instalación
12. Integración con el Proyecto Unificado SCO
13. Pruebas
14. Seguridad
15. Mantenimiento y Mejoras Futuras
16. Anexos

3. Historial de Versiones

Versión	Fecha	Autor	Descripción del Cambio
0.1.0	10/06/2026	Arquitectura	Borrador inicial de la arquitectura neuronal.
0.5.0	14/06/2026	Dev Team	Implementación del DAO, Motor IA y vistas Swing.
0.8.0	16/06/2026	Dev Team	Integración de algoritmos de decaimiento de Learning Rate.
1.0.0	18/06/2026	Documentación	Documentación exhaustiva extendida, corrección de bug MIN_VALUE, y entrega.

4. Introducción

4.1. Propósito del Documento

El presente documento tiene como objetivo proporcionar una descripción exhaustiva y profunda de la arquitectura, diseño, algoritmos matemáticos y métodos de integración del **Módulo 8 (Análisis Predictivo con IA)** del Sistema Contable Computarizado (SCO).

4.2. Alcance del Módulo

El módulo **SÍ** hace:

- Lectura histórica de los consolidados mensuales de compras y ventas de las empresas registradas.
- Entrenamiento en tiempo real de una Red Neuronal Artificial (MLP) específica para los datos de la empresa seleccionada.
- Proyección estadística de los montos de compras y ventas para el mes consecutivo al último mes histórico.
- Generación de reportes PDF autónomos y detallados con KPIs, tablas de historial y gráficas de tendencia.

El módulo **NO** hace:

- Proyecciones de flujo de caja o análisis de balances generales (se enfoca únicamente en compras y ventas).
- Inserción o modificación de datos en la base de datos (opera en modo de sólo lectura analítica `SELECT`).

4.3. Audiencia Objetivo

Este documento está dirigido a:

- **Desarrolladores Java (Backend y UI):** Para entender el MVC y los componentes de Swing.
- **Data Scientists e Ingenieros de IA:** Para auditar y optimizar el Multilayer Perceptron (MLP) implementado.
- **Database Administrators (DBA):** Para comprender el modelo de tablas mock y su integración al ecosistema MariaDB.

4.4. Definición de Términos y Acrónimos

- **SCO:** Sistema Contable Computarizado.
- **MLP:** Multilayer Perceptron (Perceptrón Multicapa), un tipo de red neuronal artificial feed-forward.
- **MSE:** Mean Squared Error (Error Cuadrático Medio), función de pérdida empleada.
- **JDBC:** Java Database Connectivity.
- **DAO:** Data Access Object.
- **POJO:** Plain Old Java Object.

5. Descripción General del Sistema

5.1. Contexto dentro del Proyecto SCO Unificado

El SCO es una suite completa de contabilidad. El Módulo 8 funciona como un anexo gerencial o "Dashboard de Inteligencia de Negocios". Extrae la data acumulada de los módulos operativos (Inventario, Facturación, CxC, CxP) y aplica algoritmos matemáticos avanzados para la toma de decisiones.

5.2. Objetivo Funcional

Facilitar a los gerentes financieros una herramienta gráfica y predictiva que, con un solo clic, evalúe la tendencia de crecimiento o decrecimiento de una empresa e infiera el comportamiento del mercado para el mes siguiente.

5.3. Diagrama de Contexto (Flujo Lógico)

1. **Módulos Operativos** -> Insertan facturas en BD.
2. **Módulo 8 (DAO)** -> Lee y consolida sumatorias mensuales.
3. **Módulo 8 (Motor IA)** -> Entrena la red y proyecta el mes N+1.
4. **Módulo 8 (Vista)** -> Presenta KPIs y permite exportar a PDF.

5.4. Restricciones y Suposiciones

- **Suposición Histórica:** Se requiere un mínimo de **3 meses continuos** de datos facturados.
- **Tecnología:** Java 11 o superior.
- **Motor BD:** MariaDB / MySQL.
- **Conectividad:** HikariCP.
- **Gráficos y PDF:** Librerías nativas java.awt, sin librerías de 3eros.

6. Arquitectura del Sistema

6.1. Patrón MVC Aplicado

El sistema obedece a una separación limpia donde:

- La vista delega al controlador.
- El controlador interroga al DAO en un hilo separado.
- El DAO retorna al controlador.
- El controlador inyecta la data al Motor IA.
- El controlador empuja los resultados de la IA a la vista.

6.2. Diagrama de Paquetes

```
com.sistemacontable.modulo8
├── .modelo                (POJOs: EmpresaItem, RegistroFinanciero,
ResultadoPrediccion)
├── .dao                  (ConexionDB, DatosFinancierosDAO)
├── .servicio             (MotorPrediccionIA, PreprocesadorDatos)
├── .vista                (VentanaAnalisis, PanelGrafica)
├── .controlador          (AnalisisControlador)
└── .util                 (ExportadorPDF, GeneradorTablasPDF,
GeneradorGraficaPDF, PlantillaPDF)
```

6.3. Diagrama de Componentes

Vista <-> Controlador <-> Servicio (IA) <-> DAO <-> Base de Datos MariaDB

6.4. Flujo General de Ejecución de Análisis

1. Clic del usuario en Analizar.
2. Interfaz gráfica desactiva botones y muestra progreso.
3. SwingWorker solicita SQL de consolidados mensuales.
4. SwingWorker provee el Dataset al preprocesador.
5. El preprocesador pasa data escalada al Perceptrón.
6. Forward y Backward pass por 5000 épocas.
7. De-escalado de datos (predicciones a dólares reales).
8. Volcado en POJO de Resultados.
9. Control retorna a hilo EDT (Event Dispatch Thread).
10. UI pinta barras y KPIs.

6.5. Tecnologías y Versiones

- Java 11+
- Swing AWT
- MariaDB JDBC 3.5.6
- HikariCP 4.0.3

7. Modelo de Datos

7.1. Diagrama Entidad-Relación

```
[ empresa_mock ] 1 ----- * [ historial_financiero_mock ]
| id_empresa      | id_registro
| nombre_comercial | id_empresa
                  | mes_numero
                  | año
                  | total_compras
                  | total_ventas
```

7.2. Descripción de Tablas

- `empresa_mock`: Registra clientes o entes jurídicos temporales para pruebas.
- `historial_financiero_mock`: Contiene transacciones de gastos e ingresos agrupados.

7.3. Script SQL y Prevención de Productos Cartesianos

Al integrar las tablas reales del SCO en el futuro, se empleará un bloque de subconsultas que evita que las uniones directas sobre meses y años generen filas duplicadas si coinciden varias facturas. (El código de esta consulta final se aprecia en el Anexo A).

7.4. Notas de Migración

Para el proyecto real, cambiar las referencias `empresa_mock` por la tabla de clientes/empresas, y usar la consulta final sobre `factura_compra_cabecera` y `factura_venta_cabecera`.

8. Diseño Detallado de Clases

A continuación se listan las clases completas del sistema con sus contratos de métodos y atributos clave. Se incluye el código fuente completo, incluyendo la corrección crítica sobre `Double.MIN_VALUE` que fue cambiada exitosamente por `-Double.MAX_VALUE` en `PreprocesadorDatos`.

8.X. Clase `Main.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8/Main.java`

```
package com.sistemacontable.modulo8;

import com.sistemacontable.modulo8.controlador.AnalisisControlador;
import com.sistemacontable.modulo8.vista.VentanaAnalisis;
import javax.swing.*;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Punto de entrada principal.
 *
 * Para integrar este módulo al proyecto unificado,
 * simplemente llama a Main.lanzar() desde el menú principal.
 */
public class Main {

    public static void main(String[] args) {
        // Ejecutar en el Event Dispatch Thread de Swing
        SwingUtilities.invokeLater(Main::lanzar);
    }

    /**
     * Instancia y muestra la ventana del módulo.
     * Llama a este método desde el menú principal del proyecto
     unificado.
     */
    public static void lanzar() {
        // 1. Crear Vista
        VentanaAnalisis vista = new VentanaAnalisis();

        // 2. Crear Controlador y conectarlo a la Vista
        AnalisisControlador controlador = new AnalisisControlador(vista);
        vista.setControlador(controlador);

        // 3. Cargar datos iniciales (empresas)
        controlador.cargarEmpresas();

        // 4. Mostrar ventana
        vista.setVisible(true);
    }
}
```


8.X. Clase `AppConfig.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8/config/AppConfig.java`

```
package com.sistemacontable.modulo8.config;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Configuración centralizada: lee credenciales y parámetros de
 * {@code config/db.properties} para evitar valores hardcodedos.
 *
 * Orden de búsqueda del archivo:
 * 1. Classpath raíz (útil al ejecutar con Ant desde /config)
 * 2. Variable de entorno DB_URL / DB_USER / DB_PASSWORD (override)
 */
public final class AppConfig {

    private static final Logger    LOGGER    =
Logger.getLogger(AppConfig.class.getName());
    private static final String CONFIG_FILE = "db.properties";
    private static final Properties props    = new Properties();

    static {
        cargarProperties();
    }

    private AppConfig() {} // Clase de utilidad - no instanciar

    // — Carga

    private static void cargarProperties() {
        try (InputStream is = AppConfig.class.getClassLoader()
.getResourceAsStream(CONFIG_FILE)) {
            if (is != null) {
                props.load(is);
            } else {
                LOGGER.warning("No se encontró '" + CONFIG_FILE + "' en
el classpath. Usando valores por defecto.");
                cargarValoresPorDefecto();
            }
        } catch (IOException e) {
            LOGGER.log(Level.SEVERE, "Error al leer configuración", e);
            cargarValoresPorDefecto();
        }
    }

    private static void cargarValoresPorDefecto() {
        props.setProperty("db.driver", "org.mariadb.jdbc.Driver");
        props.setProperty("db.url",
"jdbc:mariadb://localhost:3306/dbsco?useSSL=false&serverTimezone=UTC");
        props.setProperty("db.user", "root");
        props.setProperty("db.password", "");
    }
}
```

```

    }

// ————— Getters

/** Clase del driver JDBC (ej: {@code com.mysql.cj.jdbc.Driver}). */
public static String getDbDriver() {
    return getEnvOrProp("DB_DRIVER", "db.driver");
}

/** URL JDBC completa (ej: {@code jdbc:mysql://localhost:3306/dbSCO}). */
public static String getDbUrl() {
    return getEnvOrProp("DB_URL", "db.url");
}

/** Usuario de base de datos. */
public static String getDbUser() {
    return getEnvOrProp("DB_USER", "db.user");
}

/** Contraseña de base de datos. */
public static String getDbPassword() {
    return getEnvOrProp("DB_PASSWORD", "db.password");
}

// — Helper: variable de entorno tiene precedencia sobre properties

private static String getEnvOrProp(String envKey, String propKey) {
    String envVal = System.getenv(envKey);
    return (envVal != null && !envVal.isBlank()) ? envVal :
        props.getProperty(propKey, "");
}
}

```

8.X. Clase `AnalisisControlador.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8/controlador/AnalisisControlador.java

```
package com.sistemacontable.modulo8.controlador;

import com.sistemacontable.modulo8.dao.DatosFinancierosDAO;
import com.sistemacontable.modulo8.modelo.EmpresaItem;
import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;
import com.sistemacontable.modulo8.servicio.MotorPrediccionIA;
import com.sistemacontable.modulo8.util.ExportadorPDF;
import com.sistemacontable.modulo8.vista.VentanaAnalisis;
import java.awt.Desktop;
import java.io.File;
import java.sql.SQLException;
import java.util.List;
import javax.swing.*;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Controlador (MVC): Orquesta la interacción entre la Vista, el DAO y
 * los Servicios.
 *
 * Responsabilidades:
 * 1. Escuchar el clic del botón "Ejecutar Análisis" en la Vista.
 * 2. Pedir el historial al DAO (con el id_empresa seleccionado).
 * 3. Invocar el Motor IA para obtener la predicción.
 * 4. Enviar el resultado de vuelta a la Vista.
 * 5. Coordinar la exportación a PDF cuando el usuario lo solicita.
 */
public class AnalisisControlador {

    // — Dependencias

    private final VentanaAnalisis vista;
    private final DatosFinancierosDAO dao;
    private final MotorPrediccionIA motorIA;
    private final ExportadorPDF exportadorPDF;

    // Estado del último análisis (para exportar sin recalcular)
    private ResultadoPrediccion ultimoResultado;

    // Referencia al worker activo para poder cancelarlo
    private SwingWorker<ResultadoPrediccion, Void> workerAnalisis;

    // — Constructor

    public AnalisisControlador(VentanaAnalisis vista) {
        this.vista = vista;
        this.dao = new DatosFinancierosDAO();
        this.motorIA = new MotorPrediccionIA();
        this.exportadorPDF = new ExportadorPDF();
    }

    // — Acciones públicas (llamadas desde la Vista)
```

```

/**
 * Carga la lista de empresas disponibles y la envía a la Vista.
 * Se llama al inicializar la ventana.
 */
public void cargarEmpresas() {
    try {
        List<EmpresaItem> empresas = dao.obtenerEmpresas();
        vista.poblarSelectorEmpresas(empresas);
    } catch (SQLException e) {
        mostrarError("No se pudo conectar a la base de datos.\n" +
e.getMessage());
    }
}

/**
 * Ejecuta el análisis predictivo para la empresa seleccionada.
 * Se ejecuta en un hilo de fondo (SwingWorker) para no bloquear la
UI.
 */
 * @param idEmpresa ID de la empresa seleccionada en la Vista.
 */
public void ejecutarAnalisis(int idEmpresa) {
    // Cancelar el worker anterior si sigue corriendo
    if (workerAnalisis != null && !workerAnalisis.isDone()) {
        workerAnalisis.cancel(true);
    }

    vista.setEstadoCargando(true);
    vista.limpiarResultados();

    workerAnalisis = new SwingWorker<>() {

        @Override
        protected ResultadoPrediccion doInBackground() throws
Exception {
            // 1. Obtener historial desde la BD
            List<RegistroFinanciero> historial =
dao.obtenerHistorial(idEmpresa);

            if (historial.isEmpty()) {
                throw new IllegalStateException(
                    "No se encontró historial financiero para la
empresa seleccionada.");
            }

            // 2. Obtener nombre de la empresa
            String nombre = dao.obtenerNombreEmpresa(idEmpresa);

            // 3. Ejecutar la red neuronal
            return motorIA.predecir(historial, nombre, idEmpresa);
        }

        @Override
        protected void done() {
            vista.setEstadoCargando(false);
            try {
                ultimoResultado = get();
                vista.mostrarResultados(ultimoResultado);
            } catch (java.util.concurrent.ExecutionException ee) {
                Throwable causa = ee.getCause();
                if (causa instanceof IllegalArgumentException ||
                    causa instanceof IllegalStateException) {
                    mostrarError(causa.getMessage());
                }
            }
        }
    };
}

```

```

        } else {
            mostrarError("Error durante el análisis: " +
causa.getMessage());
        }
    } catch (Exception ex) {
        mostrarError("Error inesperado: " + ex.getMessage());
    }
}

};

workerAnalisis.execute();
}

/**
 * Exporta el último análisis a PDF.
 * Muestra un diálogo para que el usuario elija la ruta de guardado.
 */
public void exportarPDF() {
    if (ultimoResultado == null) {
        mostrarError("Primero ejecute un análisis antes de
exportar.");
        return;
    }

    JFileChooser chooser = new JFileChooser();
    chooser.setDialogTitle("Guardar Reporte PDF");
    chooser.setSelectedFile(new File("Analisis_" +
        ultimoResultado.getNombreEmpresa().replaceAll("[^a-zA-Z0-
9]", "_") + ".pdf"));

    chooser.setFileFilter(new
javax.swing.filechooser.FileNameExtensionFilter("PDF (*.pdf)", "pdf"));

    int opcion = chooser.showSaveDialog(null);
    if (opcion != JFileChooser.APPROVE_OPTION) return;

    String ruta = chooser.getSelectedFile().getAbsolutePath();
    if (!ruta.toLowerCase().endsWith(".pdf")) ruta += ".pdf";

    final String rutaFinal = ruta;
    vista.setEstadoCargando(true);

    SwingWorker<Void, Void> worker = new SwingWorker<>() {
        @Override
        protected Void doInBackground() throws Exception {
            exportadorPDF.generarPDF(ultimoResultado, rutaFinal);
            return null;
        }

        @Override
        protected void done() {
            vista.setEstadoCargando(false);
            try {
                get();
                int resp = JOptionPane.showConfirmDialog(null,
                    "PDF generado exitosamente.\n¿Desea abrirlo
ahora?",

                    "Exportación completada",
                    JOptionPane.YES_NO_OPTION,
                    JOptionPane.INFORMATION_MESSAGE);
                if (resp == JOptionPane.YES_OPTION) {
                    try {
                        Desktop.getDesktop().open(new
File(rutaFinal));

```

```
        } catch (Exception ex) {
            mostrarError("No se pudo abrir el archivo: "
+ ex.getMessage());
        }
    } catch (Exception ex) {
        mostrarError("Error al generar el PDF: " +
ex.getMessage());
    }
}

};

worker.execute();
}

// ----- Internos

private void mostrarError(String mensaje) {
    JOptionPane.showMessageDialog(null, mensaje, "Error - Módulo 8
IA",
        JOptionPane.ERROR_MESSAGE);
}
```

8.X. Clase `ConexionDB.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8\dao\ConexionDB.java`

```
package com.sistemacontable.modulo8.dao;

import com.sistemacontable.modulo8.config.AppConfig;
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

import java.sql.Connection;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * DAO: Singleton thread-safe para la conexión JDBC con pool HikariCP.
 *
 * Las credenciales se leen desde {@code config/db.properties}
 * a través de {@link AppConfig}, eliminando valores hardcodedos.
 *
 * Soporta MySQL, MariaDB y PostgreSQL según el driver configurado.
 */
public class ConexionDB {

    private static final Logger LOGGER =
        Logger.getLogger(ConexionDB.class.getName());

    // — Singleton (double-checked locking, thread-safe)

    private static volatile ConexionDB instancia;
    private HikariDataSource dataSource;

    private ConexionDB() {
        try {
            Class.forName(AppConfig.getDbDriver());
        } catch (ClassNotFoundException e) {
            LOGGER.log(Level.SEVERE, "Driver JDBC no encontrado: " +
                AppConfig.getDbDriver(), e);
        }

        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(AppConfig.getDbUrl());
        config.setUsername(AppConfig.getDbUser());
        config.setPassword(AppConfig.getDbPassword());

        // Configuraciones recomendadas para un pool robusto
        config.setMaximumPoolSize(10);
        config.setMinimumIdle(2);
        config.setConnectionTimeout(30000); // 30 segundos
        config.setIdleTimeout(600000);       // 10 minutos
        config.setMaxLifetime(1800000);     // 30 minutos

        this.dataSource = new HikariDataSource(config);
        LOGGER.info("Pool de conexiones (HikariCP) inicializado → " +
            AppConfig.getDbUrl());
    }

    public static ConexionDB getInstancia() {
```

```

        if (instancia == null) {
            synchronized (ConexionDB.class) {
                if (instancia == null) {
                    instancia = new ConexionDB();
                }
            }
        }
        return instancia;
    }
}

```

// — API pública

```

/**
 * Retorna una conexión JDBC desde el pool HikariCP.
 * HikariCP garantiza que la conexión devuelta sea válida.
 *
 * @throws SQLException si el driver no está en el classpath
 *                       o las credenciales son incorrectas.
 */
public Connection getConexion() throws SQLException {
    return dataSource.getConnection();
}

/**
 * Cierra el pool de conexiones de forma segura y limpia el Singleton
 * para permitir su recreación si es necesario.
 */
public synchronized void cerrar() {
    if (dataSource != null && !dataSource.isClosed()) {
        dataSource.close();
        LOGGER.info("Pool de conexiones HikariCP cerrado
correctamente.");
    }
    instancia = null;
}
}

```


8.X. Clase `DatosFinancierosDAO.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8\dao\DatosFinancierosDAO.java

```
package com.sistemacontable.modulo8.dao;

import com.sistemacontable.modulo8.modelo.EmpresaItem;
import com.sistemacontable.modulo8.modelo.RegistroFinanciero;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * DAO: Extrae el historial financiero de la base de datos.
 *
 * REGLA MULTI-EMPRESA: Todos los métodos reciben id_empresa
 * para garantizar el aislamiento de datos entre entidades.
 *
 * INTEGRACIÓN FUTURA: Solo hay que modificar las constantes SQL_*
 * para apuntar a las tablas reales del proyecto unificado.
 */
public class DatosFinancierosDAO {

    // — Consultas SQL (Mock - tablas provisionales)

    private static final String SQL_HISTORIAL MOCK =
        "SELECT id_empresa, mes_numero, anio, total_compras,
total_ventas " +
        "FROM historial_financiero_mock " +
        "WHERE id_empresa = ? " +
        "ORDER BY anio ASC, mes_numero ASC";

    private static final String SQL_EMPRESAS MOCK =
        "SELECT id_empresa, nombre_comercial " +
        "FROM empresa_mock " +
        "ORDER BY nombre_comercial";

    private static final String SQL_NOMBRE_EMPRESA MOCK =
        "SELECT nombre_comercial FROM empresa_mock WHERE id_empresa =
?";

    // — Consultas SQL integración futura (descomenta cuando el proyecto
se unifique)
    /**
    private static final String SQL_HISTORIAL_REAL =
        "SELECT " +
        "    v.mes_numero, " +
        "    v.anio, " +
        "    COALESCE(c.total_compras, 0) AS total_compras, " +
        "    v.total_ventas " +
        "FROM ( " +
        "    SELECT MONTH(fecha_emision) AS mes_numero,
YEAR(fecha_emision) AS anio, SUM(total_factura) AS total_ventas " +
        "    FROM ventas_facturas " +
        "    WHERE id_empresa = ? AND YEAR(fecha_emision) = 2025 " +
        "    GROUP BY YEAR(fecha_emision), MONTH(fecha_emision) " +
        ") v " +

```

```

        "LEFT JOIN (" +
            "                SELECT  MONTH(fecha_emision) AS mes_numero,
YEAR(fecha_emision) AS anio, SUM(total_factura) AS total_compras " +
            "            FROM compras_facturas " +
            "            WHERE id_empresa = ? AND YEAR(fecha_emision) = 2025 " +
            "            GROUP BY YEAR(fecha_emision), MONTH(fecha_emision) " +
            ") c " +
            "            ON v.mes_numero = c.mes_numero AND v.anio = c.anio " +
            "ORDER BY v.anio ASC, v.mes_numero ASC";
// Nota al integrar: Esta consulta requiere setear el parámetro
idEmpresa dos veces (ps.setInt(1, id); ps.setInt(2, id)).
*/

```

//

—

Dependencia

```
private final ConexionDB conexionDB;
```

```
public DatosFinancierosDAO() {
    this.conexionDB = ConexionDB.getInstance();
}

```

//

—

Métodos

públicos

```

/**
 * Retorna el historial financiero mensual de una empresa.
 *
 * @param idEmpresa Identificador de la empresa (parámetro
obligatorio Multi-Empresa).
 * @return Lista ordenada cronológicamente de registros financieros.
 * @throws SQLException si hay error de acceso a la BD.
 */
public List<RegistroFinanciero> obtenerHistorial(int idEmpresa)
throws SQLException {
    List<RegistroFinanciero> lista = new ArrayList<>();
    Connection conn = conexionDB.getConnection();

    try (PreparedStatement ps =
conn.prepareStatement(SQL_HISTORIAL MOCK)) {
        ps.setInt(1, idEmpresa);
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                RegistroFinanciero r = new RegistroFinanciero(
                    rs.getInt("id_empresa"),
                    rs.getInt("mes_numero"),
                    rs.getInt("anio"),
                    rs.getDouble("total_compras"),
                    rs.getDouble("total_ventas")
                );
                lista.add(r);
            }
        }
    }
    return lista;
}

/**
 * Retorna el listado de todas las empresas disponibles.
 * @return Mapa ID → Nombre para poblar el selector de empresa en la
UI.
 */
public List<EmpresaItem> obtenerEmpresas() throws SQLException {
    List<EmpresaItem> empresas = new ArrayList<>();

```

```

        Connection conn = conexionDB.getConnection();

        try (PreparedStatement ps =
conn.prepareStatement(SQL_EMPRESAS MOCK);
        ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                empresas.add(new EmpresaItem(
                    rs.getInt("id_empresa"),
                    rs.getString("nombre_comercial")
                ));
            }
        }
        return empresas;
    }

    /**
     * Obtiene el nombre comercial de una empresa por su ID.
     * @param idEmpresa Identificador.
     * @return Nombre comercial o "Empresa desconocida" si no existe.
     */
    public String obtenerNombreEmpresa(int idEmpresa) throws SQLException
    {
        Connection conn = conexionDB.getConnection();

        try (PreparedStatement ps =
conn.prepareStatement(SQL_NOMBRE_EMPRESA MOCK)) {
            ps.setInt(1, idEmpresa);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return rs.getString("nombre_comercial");
                }
            }
        }
        return "Empresa desconocida";
    }
}

```

8.X. Clase `EmpresaItem.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8\modelo\EmpresaItem.java`

```
package com.sistemacontable.modulo8.modelo;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * POJO para representar de manera segura una empresa en los selectores.
 */
public class EmpresaItem {
    private final int id;
    private final String nombre;

    public EmpresaItem(int id, String nombre) {
        this.id = id;
        this.nombre = nombre;
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    @Override
    public String toString() {
        return nombre; // Para que JComboBox lo muestre correctamente si
        se usara directamente
    }
}
```

8.X. Clase `RegistroFinanciero.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8\modelo\RegistroFinanciero.java

```
package com.sistemacontable.modulo8.modelo;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Modelo: Representa una fila del historial financiero mensual.
 */
public class RegistroFinanciero {

    private int    idEmpresa;
    private int    mesNumero;
    private int    anio;
    private double totalCompras;
    private double totalVentas;

    //      —      Constructores

    public RegistroFinanciero() {}

    public RegistroFinanciero(int idEmpresa, int mesNumero, int anio,
                               double totalCompras, double totalVentas) {
        this.idEmpresa    = idEmpresa;
        this.mesNumero    = mesNumero;
        this.anio          = anio;
        this.totalCompras  = totalCompras;
        this.totalVentas   = totalVentas;
    }

    //      —      Getters      /      Setters

    public int    getIdEmpresa()    { return idEmpresa; }
    public void    setIdEmpresa(int idEmpresa)    { this.idEmpresa =
idEmpresa; }

    public int    getMesNumero()    { return mesNumero; }
    public void    setMesNumero(int mesNumero)    { this.mesNumero =
mesNumero; }

    public int    getAnio()          { return anio; }
    public void    setAnio(int anio) { this.anio = anio; }

    public double getTotalCompras() { return totalCompras; }
    public void    setTotalCompras(double totalCompras) {
this.totalCompras = totalCompras; }

    public double getTotalVentas() { return totalVentas; }
    public void    setTotalVentas(double totalVentas) { this.totalVentas
= totalVentas; }

    /** Nombre corto del mes para mostrar en gráficas. */
    public String getNombreMes() {
        String[] meses = {"", "Ene", "Feb", "Mar", "Abr", "May", "Jun",
                           "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"};
        return (mesNumero >= 1 && mesNumero <= 12) ? meses[mesNumero] :
```

```
"?";  
    }  
  
    @Override  
    public String toString() {  
        return String.format("Mes %02d/%d - Ventas: %.2f | Compras:  
%.2f",  
                               mesNumero, anio, totalVentas, totalCompras);  
    }  
}
```

8.X. Clase `ResultadoPrediccion.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8\modelo\ResultadoPrediccion.java

```
package com.sistemacontable.modulo8.modelo;

import java.time.LocalDateTime;
import java.util.List;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Modelo: Encapsula el resultado completo del análisis predictivo.
 * Este objeto se pasa entre el Motor IA, el Controlador y el generador
 * de PDF.
 */
public class ResultadoPrediccion {

    private String          nombreEmpresa;
    private int             idEmpresa;
    private double          ventasPredichas;
    private double          comprasPredichas;
    private double          tendenciaPorcentaje; // % de cambio
    respecto al último mes real
    private String          clasificacionTendencia; // "CRECIMIENTO",
    "ESTABLE", "DECRECIMIENTO"
    private String          nivelConfianza; // "Alto",
    "Medio", "Bajo"
    private List<RegistroFinanciero> historial;
    private LocalDateTime   fechaAnálisis;
    private long            tiempoEntrenamientoMs;

    // — Constructor

    public ResultadoPrediccion() {
        this.fechaAnálisis = LocalDateTime.now();
    }

    // — Getters / Setters

    public String getNombreEmpresa() { return nombreEmpresa; }
    public void setNombreEmpresa(String s) { this.nombreEmpresa = s; }

    public int getIdEmpresa() { return idEmpresa; }
    public void setIdEmpresa(int id) { this.idEmpresa = id; }

    public double getVentasPredichas() { return ventasPredichas; }
    public void setVentasPredichas(double v) { this.ventasPredichas = v; }

    public double getComprasPredichas() { return comprasPredichas; }
    public void setComprasPredichas(double c) { this.comprasPredichas = c; }

    public double getTendenciaPorcentaje() { return
```

```

tendenciaPorcentaje; }
    public void      setTendenciaPorcentaje(double t)      {
this.tendenciaPorcentaje = t; }

    public String    getClasificacionTendencia()            { return
clasificacionTendencia; }
    public void      setClasificacionTendencia(String c)    {
this.clasificacionTendencia = c; }

    public String    getNivelConfianza()                    { return
nivelConfianza; }
    public void      setNivelConfianza(String n)            {
this.nivelConfianza = n; }

    public List<RegistroFinanciero> getHistorial()           { return
historial; }
    public void      setHistorial(List<RegistroFinanciero> h){
this.historial = h; }

    public LocalDateTime getFechaAnalisis()                  { return
fechaAnalisis; }
    public void      setFechaAnalisis(LocalDateTime f)      {
this.fechaAnalisis = f; }

    public long      getTiempoEntrenamientoMs()              { return
tiempoEntrenamientoMs; }
    public void      setTiempoEntrenamientoMs(long t)       {
this.tiempoEntrenamientoMs = t; }

//      —      Utilidades


---


    /** Calcula la utilidad proyectada (ventas - compras). */
    public double getUtilidadProyectada() {
        return ventasPredichas - comprasPredichas;
    }

    /** Devuelve el último registro real del historial (mes más
reciente). */
    public RegistroFinanciero getUltimoRegistro() {
        if (historial == null || historial.isEmpty()) return null;
        return historial.get(historial.size() - 1);
    }
}

```


8.X. Clase `MotorPrediccionIA.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8/servicio/MotorPrediccionIA.java

```
package com.sistemacontable.modulo8.servicio;

import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import java.util.List;
import java.util.Random;
import java.util.logging.Logger;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Servicio: Motor de predicción mediante Red Neuronal Artificial (MLP).
 *
 * Implementación pura en Java (sin dependencia externa de DL4J)
 * para maximizar la portabilidad del módulo.
 *
 * Arquitectura de la red:
 * - Capa de entrada: 3 neuronas (compras, ventas, índice temporal)
 * - Capa oculta 1: 8 neuronas, activación ReLU
 * - Capa oculta 2: 4 neuronas, activación ReLU
 * - Capa de salida: 2 neuronas, activación Sigmoid (ventas pred.,
compras pred.)
 *
 * Algoritmo de entrenamiento: Backpropagation con descenso de gradiente
estocástico.
 */
public class MotorPrediccionIA {

    private static final Logger LOGGER =
Logger.getLogger(MotorPrediccionIA.class.getName());

    // ————— Hiperparámetros

    private static final int INPUT_SIZE = 3;
    private static final int HIDDEN1_SIZE = 8;
    private static final int HIDDEN2_SIZE = 4;
    private static final int OUTPUT_SIZE = 2;
    private static final int EPOCHS = 5000;
    private static final double TASA_APRENDIZAJE_INICIAL = 0.05;

    /** Intervalo de épocas para imprimir el error (0 = sin log). */
    private static final int LOG_INTERVALO = 1000;

    // — Pesos de la red (inicializados en constructor)

    private double[][] w1; // INPUT → HIDDEN1
    private double[] b1; // bias HIDDEN1
    private double[][] w2; // HIDDEN1 → HIDDEN2
    private double[] b2; // bias HIDDEN2
    private double[][] w3; // HIDDEN2 → OUTPUT
    private double[] b3; // bias OUTPUT

    private final PreprocesadorDatos preprocesador;

    // ————— Constructor
```

```

    public MotorPrediccionIA() {
        this.preprocesador = new PreprocesadorDatos();
        inicializarPesos();
    }

    // — Inicialización de pesos

```

```

    /** Inicializa pesos con escala Xavier para convergencia más rápida.
    */
    private void inicializarPesos() {
        Random rng = new Random(42); // semilla fija para
        reproducibilidad

        w1 = new double[INPUT_SIZE][HIDDEN1_SIZE];
        b1 = new double[HIDDEN1_SIZE];
        w2 = new double[HIDDEN1_SIZE][HIDDEN2_SIZE];
        b2 = new double[HIDDEN2_SIZE];
        w3 = new double[HIDDEN2_SIZE][OUTPUT_SIZE];
        b3 = new double[OUTPUT_SIZE];

        llenarPesos(rng, w1, INPUT_SIZE);
        llenarPesos(rng, w2, HIDDEN1_SIZE);
        llenarPesos(rng, w3, HIDDEN2_SIZE);
    }

    private void llenarPesos(Random rng, double[][] w, int fanIn) {
        double escala = Math.sqrt(2.0 / fanIn);
        for (double[] fila : w) {
            for (int j = 0; j < fila.length; j++) {
                fila[j] = rng.nextGaussian() * escala;
            }
        }
    }
}

// — API pública

```

```

    /**
     * Entrena la red neuronal y genera la predicción para el mes
     * siguiente.
     *
     * @param historial Lista de registros financieros ordenados
     * cronológicamente.
     * @param nombreEmpresa Nombre de la empresa para incluir en el
     * resultado.
     * @param idEmpresa ID de la empresa (soporte Multi-Empresa).
     * @return {@link ResultadoPrediccion} con ventas y compras
     * proyectadas.
     * @throws IllegalArgumentException si el historial tiene menos de 3
     * meses.
     */
    public ResultadoPrediccion predecir(List<RegistroFinanciero>
    historial,

                                     String nombreEmpresa,
                                     int idEmpresa) {

        if (historial == null || historial.size() < 3) {
            throw new IllegalArgumentException(
                "Se necesitan al menos 3 meses de historial para
                entrenar la red neuronal.");
        }
    }
}

```

```

    }

    long inicio = System.currentTimeMillis();

    // 1. Reiniciar pesos para cada empresa (evita contaminación
entre análisis)
    inicializarPesos();

    // 2. Preprocesar: normalización Min-Max
    preprocesador.ajustar(historial);
    double[][] X = preprocesador.matrizEntrada(historial);
    double[][] Y = preprocesador.matrizSalida(historial);

    double tasaAprendizajeActual = TASA_APRENDIZAJE_INICIAL;

    // 3. Entrenamiento con log del error cada LOG_INTERVALO épocas
    for (int epoca = 0; epoca < EPOCAS; epoca++) {
        if (Thread.currentThread().isInterrupted()) {
            throw new IllegalStateException("Análisis cancelado por
el usuario.");
        }

        // Learning Rate Decay: reducir la tasa de aprendizaje un 10%
cada 1000 épocas
        // para evitar oscilaciones y permitir que la red converja de
forma más estable.
        if (epoca > 0 && epoca % 1000 == 0) {
            tasaAprendizajeActual *= 0.90;
        }

        for (int i = 0; i < X.length; i++) {
            entrenarMuestra(X[i], Y[i], tasaAprendizajeActual);
        }

        if (LOG_INTERVALO > 0 && (epoca + 1) % LOG_INTERVALO == 0) {
            double mse = calcularMSE(X, Y);
            LOGGER.info(String.format("Época %d/%d → MSE: %.6f",
epoca + 1, EPOCAS, mse));
        }
    }

    // 3.5. Obtener el MSE final para medir la calidad del ajuste
    double mseFinal = calcularMSE(X, Y);

    // 4. Inferencia: predecir el mes siguiente
    double[] vectorPrediccion =
preprocesador.vectorUltimoMes(historial);
    double[] salidaNorm = forward(vectorPrediccion);

    double ventasPred = preprocesador.desnormalizarVentas
(salidaNorm[0]);
    double comprasPred =
preprocesador.desnormalizarCompras(salidaNorm[1]);

    // 5. Calcular tendencia respecto al último mes real
    RegistroFinanciero ultimo = historial.get(historial.size() - 1);
    double tendencia = ((ventasPred - ultimo.getTotalVentas()) /
ultimo.getTotalVentas()) * 100.0;

    // 6. Ensamblar y retornar resultado
    ResultadoPrediccion resultado = new ResultadoPrediccion();
    resultado.setNombreEmpresa (nombreEmpresa);
    resultado.setIdEmpresa (idEmpresa);

```

```

        resultado.setVentasPredichas      (Math.max(0, ventasPred));
        resultado.setComprasPredichas     (Math.max(0, comprasPred));
        resultado.setTendenciaPorcentaje(tendencia);

resultado.setClasificacionTendencia(clasificarTendencia(tendencia));
                                resultado.setNivelConfianza
(calcularConfianza(historial.size(), mseFinal));
        resultado.setHistorial            (historial);
        resultado.setTiempoEntrenamientoMs(System.currentTimeMillis() -
inicio);

        long tiempoTotal = System.currentTimeMillis() - inicio;
        LOGGER.info(String.format("Predicción completada en %d ms →
Ventas: $%.2f | Compras: $%.2f",
                                tiempoTotal, ventasPred, comprasPred));

        return resultado;
    }

// — Forward Pass

/** Ejecuta una pasada hacia adelante y devuelve la salida de la red.
*/
private double[] forward(double[] entrada) {
    double[] h1 = activarReLU      (multiplicar(entrada, w1, b1));
    double[] h2 = activarReLU      (multiplicar(h1,      w2, b2));
    return      activarSigmoid     (multiplicar(h2,      w3, b3));
}

// — Backpropagation

private void entrenarMuestra(double[] x, double[] yReal, double
tasaAprendizaje) {
    // - Forward -
    double[] netH1 = multiplicar(x, w1, b1);
    double[] h1    = activarReLU(netH1);
    double[] netH2 = multiplicar(h1, w2, b2);
    double[] h2    = activarReLU(netH2);
    double[] netOut= multiplicar(h2, w3, b3);
    double[] out   = activarSigmoid(netOut);

    // - Delta capa de salida -
    double[] deltaOut = new double[OUTPUT_SIZE];
    for (int k = 0; k < OUTPUT_SIZE; k++) {
        double error = yReal[k] - out[k];
        deltaOut[k] = error * derivadaSigmoid(out[k]);
    }

    // - Delta capa oculta 2 -
    double[] deltaH2 = new double[HIDDEN2_SIZE];
    for (int j = 0; j < HIDDEN2_SIZE; j++) {
        double suma = 0;
        for (int k = 0; k < OUTPUT_SIZE; k++) suma += deltaOut[k] *
w3[j][k];
        deltaH2[j] = suma * derivadaReLU(netH2[j]);
    }

    // - Delta capa oculta 1 -
    double[] deltaH1 = new double[HIDDEN1_SIZE];
    for (int j = 0; j < HIDDEN1_SIZE; j++) {
        double suma = 0;

```

```

        for (int k = 0; k < HIDDEN2_SIZE; k++) suma += deltaH2[k] *
w2[j][k];
        deltaH1[j] = suma * derivadaReLU(netH1[j]);
    }

    // - Actualizar W3 y b3 -
    for (int j = 0; j < HIDDEN2_SIZE; j++)
        for (int k = 0; k < OUTPUT_SIZE; k++)
            w3[j][k] += tasaAprendizaje * deltaOut[k] * h2[j];
    for (int k = 0; k < OUTPUT_SIZE; k++)
        b3[k] += tasaAprendizaje * deltaOut[k];

    // - Actualizar W2 y b2 -
    for (int j = 0; j < HIDDEN1_SIZE; j++)
        for (int k = 0; k < HIDDEN2_SIZE; k++)
            w2[j][k] += tasaAprendizaje * deltaH2[k] * h1[j];
    for (int k = 0; k < HIDDEN2_SIZE; k++)
        b2[k] += tasaAprendizaje * deltaH2[k];

    // - Actualizar W1 y b1 -
    for (int i = 0; i < INPUT_SIZE; i++)
        for (int k = 0; k < HIDDEN1_SIZE; k++)
            w1[i][k] += tasaAprendizaje * deltaH1[k] * x[i];
    for (int k = 0; k < HIDDEN1_SIZE; k++)
        b1[k] += tasaAprendizaje * deltaH1[k];
}

// — Cálculo de error MSE (diagnóstico)

```

```

private double calcularMSE(double[][] X, double[][] Y) {
    double errorTotal = 0;
    for (int i = 0; i < X.length; i++) {
        double[] pred = forward(X[i]);
        for (int k = 0; k < OUTPUT_SIZE; k++) {
            double diff = Y[i][k] - pred[k];
            errorTotal += diff * diff;
        }
    }
    return errorTotal / (X.length * OUTPUT_SIZE);
}

```

// — Funciones matemáticas

```

private double[] multiplicar(double[] entrada, double[][] pesos,
double[] bias) {
    int salida = pesos[0].length;
    double[] resultado = new double[salida];
    for (int j = 0; j < salida; j++) {
        resultado[j] = bias[j];
        for (int i = 0; i < entrada.length; i++) {
            resultado[j] += entrada[i] * pesos[i][j];
        }
    }
    return resultado;
}

private double[] activarReLU(double[] z) {
    double[] a = new double[z.length];
    for (int i = 0; i < z.length; i++) a[i] = Math.max(0, z[i]);
    return a;
}

```

```

    private double[] activarSigmoid(double[] z) {
        double[] a = new double[z.length];
        for (int i = 0; i < z.length; i++) a[i] = 1.0 / (1.0 + Math.exp(-
z[i]));
        return a;
    }

    private double derivadaReLU    (double z) { return z > 0 ? 1.0 : 0.0;
}
    private double derivadaSigmoid(double a) { return a * (1.0 - a); }

//      —      Clasificadores

```

```

    private String clasificarTendencia(double porcentaje) {
        if (porcentaje > 5.0) return "CRECIMIENTO";
        if (porcentaje < -5.0) return "DECRECIMIENTO";
        return "ESTABLE";
    }

    private String calcularConfianza(int nMeses, double mseFinal) {
        // Base según historial
        int scoreMeses = 0;
        if (nMeses >= 10) scoreMeses = 3;
        else if (nMeses >= 6) scoreMeses = 2;
        else scoreMeses = 1;

        // Base según error cuadrático medio (normalizado)
        int scoreMse = 0;
        if (mseFinal < 0.05) scoreMse = 3;          // Excelente convergencia
        else if (mseFinal < 0.15) scoreMse = 2;      // Convergencia aceptable
        else scoreMse = 1;                          // Pobre convergencia

        int scoreTotal = scoreMeses + scoreMse;

        if (scoreTotal >= 5) return "Alto";
        if (scoreTotal >= 3) return "Medio";
        return "Bajo";
    }
}

```

8.X. Clase `PreprocesadorDatos.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8\servicio\PreprocesadorDatos.java

```
package com.sistemacontable.modulo8.servicio;

import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import java.util.List;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Servicio: Preprocesamiento y normalización de datos para la red
 * neuronal.
 *
 * La red neuronal trabaja mejor con valores en el rango [0, 1].
 * Esta clase escala los datos usando Min-Max normalization y los
 * organiza en matrices de entrada (features) y salida (labels).
 *
 * FEATURES de entrada por mes:
 * [0] total_compras normalizado
 * [1] total_ventas normalizado
 * [2] índice de mes normalizado (posición temporal)
 *
 * LABEL de salida por mes:
 * [0] total_ventas del mes siguiente (normalizado)
 * [1] total_compras del mes siguiente (normalizado)
 */
public class PreprocesadorDatos {

    // Estado interno

    private double minVentas, maxVentas;
    private double minCompras, maxCompras;
    private boolean estadoCalculado = false;

    // API pública

    /**
     * Calcula min/max del historial y normaliza todos los valores.
     * Debe llamarse antes de usar matrizEntrada() o matrizSalida().
     *
     * @param historial Lista de registros ordenados cronológicamente.
     */
    public void ajustar(List<RegistroFinanciero> historial) {
        if (historial == null || historial.isEmpty()) {
            throw new IllegalArgumentException("El historial no puede
            estar vacío.");
        }

        minVentas = Double.MAX_VALUE;
        maxVentas = -Double.MAX_VALUE;
        minCompras = Double.MAX_VALUE;
        maxCompras = -Double.MAX_VALUE;

        for (RegistroFinanciero r : historial) {
            if (r.getTotalVentas() < minVentas) minVentas =
            r.getTotalVentas();
            if (r.getTotalVentas() > maxVentas) maxVentas =

```

```

r.getTotalVentas();
        if (r.getTotalCompras() < minCompras) minCompras =
r.getTotalCompras();
        if (r.getTotalCompras() > maxCompras) maxCompras =
r.getTotalCompras();
    }

    // Evitar división por cero si todos los valores son iguales
    if (maxVentas == minVentas) maxVentas = minVentas + 1;
    if (maxCompras == minCompras) maxCompras = minCompras + 1;

    estadoCalculado = true;
}

/**
 * Construye la matriz de entrada para la red neuronal.
 * Cada fila = un mes de entrenamiento (todos excepto el último).
 * Cada columna = una feature normalizada.
 *
 * @param historial Lista de registros.
 * @return double[n-1][3]
 */
public double[][] matrizEntrada(List<RegistroFinanciero> historial) {
    verificarEstado();
    int n = historial.size();
    double[][] matriz = new double[n - 1][3];
    int total = historial.size();

    for (int i = 0; i < n - 1; i++) {
        RegistroFinanciero r = historial.get(i);
        matriz[i][0] = normalizar(r.getTotalCompras(), minCompras,
maxCompras);
        matriz[i][1] = normalizar(r.getTotalVentas(), minVentas,
maxVentas);
        matriz[i][2] = (double) i / (total - 1); // posición
temporal
    }
    return matriz;
}

/**
 * Construye la matriz de salida (labels) para la red neuronal.
 * La salida del mes i es el valor real del mes i+1.
 *
 * @param historial Lista de registros.
 * @return double[n-1][2]
 */
public double[][] matrizSalida(List<RegistroFinanciero> historial) {
    verificarEstado();
    int n = historial.size();
    double[][] matriz = new double[n - 1][2];

    for (int i = 0; i < n - 1; i++) {
        RegistroFinanciero siguiente = historial.get(i + 1);
        matriz[i][0] = normalizar(siguiente.getTotalVentas(),
minVentas, maxVentas);
        matriz[i][1] = normalizar(siguiente.getTotalCompras(),
minCompras, maxCompras);
    }
    return matriz;
}

/**

```



```

        * Construye el vector de entrada del último mes (para predecir el
siguiente).
    *
    * @param historial Lista de registros.
    * @return double[3]
    */
    public double[] vectorUltimoMes(List<RegistroFinanciero> historial) {
        verificarEstado();
        RegistroFinanciero ultimo = historial.get(historial.size() - 1);
        return new double[]{
            normalizar(ultimo.getTotalCompras(), minCompras,
maxCompras),
            normalizar(ultimo.getTotalVentas(), minVentas,
maxVentas),
            1.0 // posición temporal = 1.0 (próximo mes)
        };
    }

    /**
    * Desnormaliza un valor de ventas (vuelve a la escala original en
$).
    * @param valorNorm
    */
    public double desnormalizarVentas(double valorNorm) {
        return valorNorm * (maxVentas - minVentas) + minVentas;
    }

    /**
    * Desnormaliza un valor de compras.
    * @return
    */
    public double desnormalizarCompras(double valorNorm) {
        return valorNorm * (maxCompras - minCompras) + minCompras;
    }
}

// — Internos

private double normalizar(double valor, double min, double max) {
    return (valor - min) / (max - min);
}

private void verificarEstado() {
    if (!estadoCalculado) {
        throw new IllegalStateException("Debe llamar a ajustar()
antes de transformar datos.");
    }
}

// — Getters para debug / reporte

public double getMinVentas() { return minVentas; }
public double getMaxVentas() { return maxVentas; }
public double getMinCompras() { return minCompras; }
public double getMaxCompras() { return maxCompras; }
}

```

8.X. Clase `ExportadorPDF.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8/util/ExportadorPDF.java`

```
package com.sistemacontable.modulo8.util;

import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import java.awt.*;
import java.awt.geom.RoundRectangle2D;
import java.awt.image.BufferedImage;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import static com.sistemacontable.modulo8.util.PlantillaPDF.*;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Utilidad: Genera un reporte PDF del análisis predictivo.
 *
 * Actúa como coordinador delegando:
 * - Dibujo de gráficas a GeneradorGraficaPDF
 * - Dibujo de tablas a GeneradorTablasPDF
 * - Configuración, paleta y empaquetado PDF a PlantillaPDF
 */
public class ExportadorPDF {

    /**
     * Genera el PDF del análisis y lo guarda en la ruta indicada.
     *
     * @param resultado Resultado del análisis predictivo.
     * @param rutaArchivo Ruta completa del PDF a crear.
     * @throws IOException si hay error de escritura en disco.
     */
    public void generarPDF(ResultadoPrediccion resultado, String
rutaArchivo) throws IOException {
        List<BufferedImage> paginas = renderizarPaginas(resultado);
        try (FileOutputStream fos = new FileOutputStream(rutaArchivo)) {
            PlantillaPDF.escribirPDF(fos, paginas);
        }
    }

    private List<BufferedImage> renderizarPaginas(ResultadoPrediccion
res) {
        List<BufferedImage> lista = new ArrayList<>();
        lista.add(renderizarPagina1(res));
        lista.add(renderizarPagina2(res));
        return lista;
    }

    private BufferedImage renderizarPagina1(ResultadoPrediccion res) {
        BufferedImage img = PlantillaPDF.nuevaPagina();
        Graphics2D g = PlantillaPDF.crearGraphics(img);

        int y = dibujarEncabezado(g, res);
        y += 22;
        y = dibujarResumenEjecutivo(g, res, y);
        y += 22;
    }
}
```

```

        GeneradorTablasPDF.dibujarTablaHistorial(g, res, y);
        PlantillaPDF.dibujarPieDePagina(g, 1, 2, res);

        g.dispose();
        return img;
    }

    private BufferedImage renderizarPagina2(ResultadoPrediccion res) {
        BufferedImage img = PlantillaPDF.nuevaPagina();
        Graphics2D g = PlantillaPDF.crearGraphics(img);

        int y = MARGIN + 10;
        PlantillaPDF.dibujarTituloSeccion(g, "Análisis de Tendencia y
Gráfica Histórica", y);
        y += 30;
        y = GeneradorGraficaPDF.dibujarGrafica(g, res, y);
        y += 22;
        dibujarRecomendaciones(g, res, y);
        PlantillaPDF.dibujarPieDePagina(g, 2, 2, res);

        g.dispose();
        return img;
    }

    private int dibujarEncabezado(Graphics2D g, ResultadoPrediccion res)
    {
        GradientPaint gp = new GradientPaint(0, 0, C_PRIMARIO, PAGE_W,
80, new Color(49, 112, 204));
        g.setPaint(gp);
        g.fillRect(0, 0, PAGE_W, 80);

        g.setColor(Color.WHITE);
        g.setFont(new Font("SansSerif", Font.BOLD, 17));
        g.drawString("SISTEMA CONTABLE COMPUTARIZADO", MARGIN, 28);

        g.setFont(new Font("SansSerif", Font.PLAIN, 10));
        g.drawString("Módulo 8 - Análisis Predictivo con Inteligencia
Artificial", MARGIN, 44);

        String fecha = "Emitido: " +
res.getFechaAnalisis().format(FMT_FECHA);
        FontMetrics fm = g.getFontMetrics();
        g.drawString(fecha, PAGE_W - MARGIN - fm.stringWidth(fecha), 44);

        g.setColor(C_AMARILLO);
        g.fillRect(0, 76, PAGE_W, 4);

        int y = 100;
        g.setColor(C_PRIMARIO);
        g.setFont(new Font("SansSerif", Font.BOLD, 13));
        g.drawString("Empresa: " + res.getNombreEmpresa(), MARGIN, y);

        y += 17;
        g.setColor(C_GRIS);
        g.setFont(new Font("SansSerif", Font.PLAIN, 10));
        g.drawString("ID: " + res.getIdEmpresa() +
" | Meses analizados: " + res.getHistorial().size() +
" | Tiempo de entrenamiento: " +
res.getTiempoEntrenamientoMs() + " ms",
MARGIN, y);

        return y + 10;
    }
}

```

```

        private int dibujarResumenEjecutivo(Graphics2D g, ResultadoPrediccion
res, int y) {
            PlantillaPDF.dibujarTituloSeccion(g, "Resumen Ejecutivo de
Predicción", y);
            y += 28;

            int cardW = (CONTENT_W - 20) / 3;
            int cardH = 68;

            dibujarKPI(g, MARGIN, y, cardW, cardH,
                "Ventas Proyectadas (Próx. Mes)",
                String.format("$%,.2f", res.getVentasPredichas()),
C_VERDE);

            dibujarKPI(g, MARGIN + cardW + 10, y, cardW, cardH,
                "Compras Proyectadas (Próx. Mes)",
                String.format("$%,.2f", res.getComprasPredichas()),
C_SECUNDARIO);

            double utilidad = res.getUtilidadProyectada();
            Color colorUtil = utilidad >= 0 ? C_VERDE : C_ROJO;
            dibujarKPI(g, MARGIN + (cardW + 10) * 2, y, cardW, cardH,
                "Utilidad Proyectada",
                String.format("$%,.2f", utilidad), colorUtil);

            y += cardH + 14;
            dibujarIndicadoresSecundarios(g, res, y);
            return y + 55;
        }

        private void dibujarKPI(Graphics2D g, int x, int y, int w, int h,
            String etiqueta, String valor, Color color)
        {
            GradientPaint gp = new GradientPaint(x, y, color, x, y + h,
color.darker());
            g.setPaint(gp);
            g.fill(new RoundRectangle2D.Double(x, y, w, h, 8, 8));

            g.setColor(new Color(255, 255, 255, 170));
            g.setFont(new Font("SansSerif", Font.PLAIN, 9));
            g.drawString(etiqueta, x + 8, y + 16);

            g.setColor(Color.WHITE);
            g.setFont(new Font("SansSerif", Font.BOLD, 15));
            g.drawString(valor, x + 8, y + h - 13);
        }

        private void dibujarIndicadoresSecundarios(Graphics2D g,
ResultadoPrediccion res, int y) {
            int cardW = (CONTENT_W - 10) / 2;
            int cardH = 44;

            PlantillaPDF.dibujarTarjetaInfo(g, MARGIN, y, cardW, cardH,
                "Clasificación de Tendencia",
                res.getClasificacionTendencia(),
PlantillaPDF.colorPorClasificacion(res.getClasificacionTendencia()));

            PlantillaPDF.dibujarTarjetaInfo(g, MARGIN + cardW + 10, y, cardW,
cardH,
                "Nivel de Confianza del Modelo",
                res.getNivelConfianza() + " (" +

```

```

res.getHistorial().size() + " meses de datos)",
    C_SECUNDARIO);

    y += cardH + 8;
    String varTexto = String.format("Variación vs. último mes real:
%.2f%%",
        res.getTendenciaPorcentaje());
    g.setFont(new Font("SansSerif", Font.BOLD, 11));
    g.setColor(res.getTendenciaPorcentaje() >= 0 ? C_VERDE : C_ROJO);
    g.drawString(varTexto, MARGIN, y + 14);
}

private int dibujarRecomendaciones(Graphics2D g, ResultadoPrediccion
res, int y) {
    PlantillaPDF.dibujarTituloSeccion(g, "Interpretación y
Recomendaciones", y);
    y += 28;

    String[] recomendaciones = generarRecomendaciones(res);
    int alturaBloque = 12 + recomendaciones.length * 20;

    g.setColor(C_ACENTO);
    g.fillRoundRect(MARGIN, y, CONTENT_W, alturaBloque, 6, 6);
    g.setColor(C_SECUNDARIO);
    g.setStroke(new BasicStroke(1.5f));
    g.drawRoundRect(MARGIN, y, CONTENT_W, alturaBloque, 6, 6);
    g.setStroke(new BasicStroke(1));

    y += 16;
    g.setFont(new Font("SansSerif", Font.PLAIN, 10));
    for (String rec : recomendaciones) {
        g.setColor(Color.DARK_GRAY);
        g.drawString(" • " + rec, MARGIN + 8, y);
        y += 20;
    }
    return y + 10;
}

private String[] generarRecomendaciones(ResultadoPrediccion res) {
    String clasificacion = res.getClasificacionTendencia();
    double tendencia = res.getTendenciaPorcentaje();
    double utilidad = res.getUtilidadProyectada();

    List<String> lista = new ArrayList<>();
    switch (clasificacion) {
        case "CRECIMIENTO":
            lista.add(String.format("Las ventas proyectan un
crecimiento del %.1f%% respecto al mes anterior.", tendencia));
            lista.add("Considerar aumentar el inventario y la
capacidad operativa para sostener la demanda.");
            lista.add("Evaluar si el flujo de caja actual soporta el
crecimiento proyectado.");
            break;
        case "DECRECIMIENTO":
            lista.add(String.format("Las ventas proyectan una caída
del %.1f%%, requiere atención inmediata.", Math.abs(tendencia)));
            lista.add("Revisar estrategias de ventas, precios y
retención de clientes.");
            lista.add("Reducir gastos no esenciales para proteger la
utilidad operativa.");
            break;
        default:
            lista.add("Las ventas se mantienen estables; el mercado

```

```
muestra comportamiento predecible.");
        lista.add("Mantener la estrategia actual y buscar
oportunidades de crecimiento moderado.");
    }
    if (utilidad < 0)
        lista.add("ALERTA: La utilidad proyectada es negativa.
Revisar estructura de costos urgentemente.");
    if ("Bajo".equals(res.getNivelConfianza()))
        lista.add("NOTA: Con menos de 6 meses de datos el modelo
tiene menor precisión. Agregar más historial mejorará las
predicciones.");

    return lista.toArray(new String[0]);
}
}
```

8.X. Clase `GeneradorGraficaPDF.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8/util/GeneradorGraficaPDF.java

```
package com.sistemacontable.modulo8.util;

import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import java.awt.*;
import java.util.List;

import static com.sistemacontable.modulo8.util.PlantillaPDF.*;

public class GeneradorGraficaPDF {

    public static int dibujarGrafica(Graphics2D g, ResultadoPrediccion
res, int y) {
        List<RegistroFinanciero> hist = res.getHistorial();
        int grafH = 200, grafW = CONTENT_W;
        int grafX = MARGIN, grafY = y;

        // Fondo
        g.setColor(new Color(248, 250, 252));
        g.fillRect(grafX, grafY, grafW, grafH);
        g.setColor(C_LINEA);
        g.drawRect(grafX, grafY, grafW, grafH);

        double maxVal = 0;
        for (RegistroFinanciero r : hist)
            maxVal = Math.max(maxVal, Math.max(r.getTotalVentas(),
r.getTotalCompras()));
        maxVal = Math.max(maxVal, res.getVentasPredichas()) * 1.15;

        int n = hist.size() + 1;
        int barMg = 8;
        int grupW = (grafW - 40) / n;
        int barW = Math.max(8, (grupW - barMg * 2) / 2);

        // Cuadrícula
        g.setFont(new Font("SansSerif", Font.PLAIN, 8));
        for (int q = 0; q <= 4; q++) {
            int lineY = grafY + grafH - (int)(grafH * q / 4.0);
            g.setColor(C_LINEA);
            g.drawLine(grafX + 30, lineY, grafX + grafW, lineY);
            g.setColor(C_GRIS);
            g.drawString(String.format("%.0f", maxVal * q / 4.0), grafX
+ 2, lineY + 4);
        }

        boolean multiYear = false;
        if (!hist.isEmpty()) {
            int firstYear = hist.get(0).getAnio();
            for (RegistroFinanciero r : hist) {
                if (r.getAnio() != firstYear) {
                    multiYear = true;
                    break;
                }
            }
        }
    }
}
```

```

    }

    // Barras históricas con gradiente
    for (int i = 0; i < hist.size(); i++) {
        RegistroFinanciero r = hist.get(i);
        int baseX = grafX + 35 + i * grupW;

        int hC = (int)(grafH * r.getTotalCompras() / maxVal);
        int hV = (int)(grafH * r.getTotalVentas() / maxVal);

        // Compras (azul)
        GradientPaint gpC = new GradientPaint(
            baseX + barMg, grafY + grafH - hC,
C_SECUNDARIO.darker(),
            baseX + barMg, grafY + grafH, C_SECUNDARIO);
        g.setPaint(gpC);
        g.fillRect(baseX + barMg, grafY + grafH - hC, barW, hC);

        // Ventas (verde)
        GradientPaint gpV = new GradientPaint(
            baseX + barMg + barW + 2, grafY + grafH - hV,
C_VERDE.darker(),
            baseX + barMg + barW + 2, grafY + grafH,
C_VERDE);
        g.setPaint(gpV);
        g.fillRect(baseX + barMg + barW + 2, grafY + grafH - hV,
barW, hV);

        g.setColor(C_GRIS);
        g.setFont(new Font("SansSerif", Font.PLAIN, 8));
        String mes = r.getNombreMes();
        if (multiYear) {
            mes += String.format(" '%02d", r.getAnio() % 100);
        }
        g.drawString(mes, baseX + barMg, grafY + grafH + 12);
    }

    // Barra de predicción (ámbar)
    int predX = grafX + 35 + hist.size() * grupW;
    int altPredV = (int)(grafH * res.getVentasPredichas() / maxVal);
    int altPredC = (int)(grafH * res.getComprasPredichas() / maxVal);

    g.setColor(new Color(254, 243, 199, 120));
    g.fillRect(predX, grafY, grupW, grafH);

    GradientPaint gpPC = new GradientPaint(predX + barMg, grafY +
grafH - altPredC,
            C_AMARILLO.darker(), predX + barMg, grafY + grafH,
C_AMARILLO);
    g.setPaint(gpPC);
    g.fillRect(predX + barMg, grafY + grafH - altPredC, barW,
altPredC);

    GradientPaint gpPV = new GradientPaint(predX + barMg + barW + 2,
grafY + grafH - altPredV,
            new Color(0, 150, 50), predX + barMg + barW + 2, grafY +
grafH, new Color(0, 200, 80));
    g.setPaint(gpPV);
    g.fillRect(predX + barMg + barW + 2, grafY + grafH - altPredV,
barW, altPredV);

    g.setColor(C_GRIS);
    g.setFont(new Font("SansSerif", Font.BOLD, 8));

```



```
        g.drawString("Pred.", predX + barMg, grafY + grafH + 12);

        // Leyenda
        y = grafY + grafH + 22;
        g.setFont(new Font("SansSerif", Font.PLAIN, 9));
        g.setPaint(C_SECUNDARIO); g.fillRect(MARGIN, y, 12, 10);
        g.setColor(Color.DARK_GRAY); g.drawString("Compras", MARGIN +
16, y + 9);
        g.setPaint(C_VERDE); g.fillRect(MARGIN + 75, y, 12, 10);
        g.setColor(Color.DARK_GRAY); g.drawString("Ventas", MARGIN +
91, y + 9);
        g.setPaint(C_AMARILLO); g.fillRect(MARGIN + 145, y, 12, 10);
        g.setColor(Color.DARK_GRAY); g.drawString("Predicción", MARGIN +
161, y + 9);

        return y + 20;
    }
}
```

8.X. Clase `GeneradorTablasPDF.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8\util\GeneradorTablasPDF.java

```
package com.sistemacontable.modulo8.util;

import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import java.awt.*;
import java.util.List;

import static com.sistemacontable.modulo8.util.PlantillaPDF.*;

public class GeneradorTablasPDF {

    public static int dibujarTablaHistorial(Graphics2D g,
    ResultadoPrediccion res, int y) {
        PlantillaPDF.dibujarTituloSeccion(g, "Historial Financiero
    Mensual", y);
        y += 28;

        List<RegistroFinanciero> hist = res.getHistorial();
        int[] colX = {MARGIN, MARGIN + 58, MARGIN + 155, MARGIN + 275,
    MARGIN + 385};
        String[] cabeceras = {"Mes", "Período", "Total Compras ($)",
    "Total Ventas ($)", "Utilidad ($)"};
        int rowH = 22;

        // Cabecera
        g.setColor(C_PRIMARIO);
        g.fillRect(MARGIN, y, CONTENT_W, rowH);
        g.setColor(Color.WHITE);
        g.setFont(new Font("SansSerif", Font.BOLD, 10));
        for (int i = 0; i < cabeceras.length; i++) {
            g.drawString(cabeceras[i], colX[i] + 4, y + 15);
        }
        y += rowH;

        // Filas de datos
        g.setFont(new Font("SansSerif", Font.PLAIN, 10));
        for (int i = 0; i < hist.size(); i++) {
            RegistroFinanciero r = hist.get(i);
            g.setColor(i % 2 == 0 ? Color.WHITE : C_ACENTO);
            g.fillRect(MARGIN, y, CONTENT_W, rowH);

            g.setColor(Color.DARK_GRAY);
            g.drawString(r.getNombreMes(),
    colX[0] + 4, y + 15);
            g.drawString(r.getMesNumero() + "/" + r.getAnio(),
    colX[1] + 4, y + 15);
            g.drawString(String.format("%.2f", r.getTotalCompras()),
    colX[2] + 4, y + 15);
            g.drawString(String.format("%.2f", r.getTotalVentas()),
    colX[3] + 4, y + 15);
            double util = r.getTotalVentas() - r.getTotalCompras();
            g.setColor(util >= 0 ? C_VERDE : C_ROJO);
            g.drawString(String.format("%.2f", util),
    colX[4] + 4, y + 15);
        }
    }
}
```

```

        y += rowH;
        g.setColor(C_LINEA);
        g.drawLine(MARGIN, y, MARGIN + CONTENT_W, y);
    }

    // Fila de predicción
    y += 4;
    g.setColor(new Color(219, 234, 254));
    g.fillRoundRect(MARGIN, y, CONTENT_W, rowH + 2, 4, 4);
    g.setColor(C_PRIMARIO);
    g.setFont(new Font("SansSerif", Font.BOLD, 10));
    g.drawString("PREDICCIÓN →",
colX[0] + 4, y + 16);
    g.drawString("Próx. mes",
colX[1] + 4, y + 16);
    g.setColor(C_SECUNDARIO);
    g.drawString(String.format("%.2f", res.getComprasPredichas()),
colX[2] + 4, y + 16);
    g.drawString(String.format("%.2f", res.getVentasPredichas()),
colX[3] + 4, y + 16);
    double utilProy = res.getUtilidadProyectada();
    g.setColor(utilProy >= 0 ? C_VERDE : C_ROJO);
    g.drawString(String.format("%.2f", utilProy),
colX[4] + 4, y + 16);

    return y + rowH + 10;
}
}

```

8.X. Clase `PlantillaPDF.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8\util\PlantillaPDF.java`

```
package com.sistemacontable.modulo8.util;

import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import java.awt.*;
import java.awt.geom.RoundRectangle2D;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

/**
 * Plantilla principal para documentos PDF.
 * Contiene paletas de colores, márgenes, métodos comunes de dibujo
 * y la lógica de bajo nivel para codificar y guardar el archivo PDF.
 */
public class PlantillaPDF {

    public static final int PAGE_W      = 595;
    public static final int PAGE_H      = 842;
    public static final int MARGIN      = 48;
    public static final int CONTENT_W   = PAGE_W - 2 * MARGIN;
    public static final int SCALE       = 4;

    public static final Color C_PRIMARIO   = new Color(30, 80, 160);
    public static final Color C_SECUNDARIO = new Color(59, 130, 246);
    public static final Color C_ACENTO     = new Color(240, 245, 255);
    public static final Color C_VERDE      = new Color(22, 163, 74);
    public static final Color C_ROJO       = new Color(220, 38, 38);
    public static final Color C_AMARILLO   = new Color(234, 179, 8);
    public static final Color C_GRIS       = new Color(100, 116, 139);
    public static final Color C_LINEA      = new Color(226, 232, 240);

    public static final DateTimeFormatter FMT_FECHA =
        DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");

    public static BufferedImage nuevaPagina() {
        return new BufferedImage(PAGE_W * SCALE, PAGE_H * SCALE,
            BufferedImage.TYPE_INT_RGB);
    }

    public static Graphics2D crearGraphics(BufferedImage img) {
        Graphics2D g = img.createGraphics();
        g.scale(SCALE, SCALE);
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
            RenderingHints.VALUE_TEXT_ANTIALIAS_LCD_HRGB);
        g.setRenderingHint(RenderingHints.KEY_RENDERING,
            RenderingHints.VALUE_RENDER_QUALITY);
        g.setColor(Color.WHITE);
        g.fillRect(0, 0, PAGE_W, PAGE_H);
    }
}
```

```

        return g;
    }

    public static Color colorPorClasificacion(String c) {
        return switch (c) {
            case "CRECIMIENTO"    -> C_VERDE;
            case "DECRECIMIENTO" -> C_ROJO;
            default                -> C_AMARILLO;
        };
    }

    public static void dibujarTituloSeccion(Graphics2D g, String titulo,
int y) {
        g.setColor(C_PRIMARIO);
        g.setFont(new Font("SansSerif", Font.BOLD, 12));
        g.drawString(titulo, MARGIN, y + 14);
        GradientPaint gp = new GradientPaint(MARGIN, 0, C_SECUNDARIO,
MARGIN + CONTENT_W, 0, C_ACENTO);
        g.setPaint(gp);
        g.fillRect(MARGIN, y + 17, CONTENT_W, 2);
    }

    public static void dibujarTarjetaInfo(Graphics2D g, int x, int y, int
w, int h,
                                     String etiqueta, String valor,
Color color) {
        g.setColor(C_ACENTO);
        g.fill(new RoundRectangle2D.Double(x, y, w, h, 6, 6));
        g.setColor(color);
        g.setStroke(new BasicStroke(1.8f));
        g.draw(new RoundRectangle2D.Double(x, y, w, h, 6, 6));
        g.setStroke(new BasicStroke(1));

        g.setColor(C_GRIS);
        g.setFont(new Font("SansSerif", Font.PLAIN, 9));
        g.drawString(etiqueta, x + 8, y + 14);

        g.setColor(color);
        g.setFont(new Font("SansSerif", Font.BOLD, 12));
        g.drawString(valor, x + 8, y + h - 10);
    }

    public static void dibujarPieDePagina(Graphics2D g, int pagActual,
int pagTotal,
                                     ResultadoPrediccion res) {
        int pieY = PAGE_H - 36;
        g.setColor(C_LINEA);
        g.fillRect(0, pieY, PAGE_W, 1);

        g.setFont(new Font("SansSerif", Font.PLAIN, 8));
        g.setColor(C_GRIS);
        g.drawString("Módulo 8 - Análisis Predictivo con IA | " +
                                     "Generado el " +
res.getFechaAnalisis().format(FMT_FECHA), MARGIN, pieY + 18);

        String pagTexto = "Página " + pagActual + " de " + pagTotal;
        FontMetrics fm = g.getFontMetrics();
        g.drawString(pagTexto, PAGE_W - MARGIN -
fm.stringWidth(pagTexto), pieY + 18);
    }

    public static void escribirPDF(OutputStream out, List<BufferedImage>
paginas) throws IOException {

```

```

List<byte[]> jpegs = new ArrayList<>();
for (BufferedImage img : paginas) {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    javax.imageio.ImageIO.write(img, "jpeg", baos);
    jpegs.add(baos.toByteArray());
}

byte[] header = "%PDF-
1.4\n".getBytes(StandardCharsets.ISO_8859_1);
int headerLen = header.length;

ByteArrayOutputStream body = new ByteArrayOutputStream();
List<Integer> offsets = new ArrayList<>();

java.util.function.Consumer<String> writeStr = s -> {
    try { body.write(s.getBytes(StandardCharsets.ISO_8859_1)); }
    catch (IOException ex) { throw new RuntimeException(ex); }
};
java.util.function.Consumer<byte[]> writeBytes = b -> {
    try { body.write(b); }
    catch (IOException ex) { throw new RuntimeException(ex); }
};

offsets.add(headerLen + body.size());
writeStr.accept("1 0 obj\n<< /Type /Catalog /Pages 2 0 R
>>\nendobj\n");

StringBuilder kids = new StringBuilder();
int firstPageObj = 3;
for (int i = 0; i < paginas.size(); i++) {
    kids.append(firstPageObj + i * 3).append(" 0 R ");
}
offsets.add(headerLen + body.size());
writeStr.accept("2 0 obj\n<< /Type /Pages /Kids [" + kids + "
/Count "
    + paginas.size() + " >>\nendobj\n");

for (int p = 0; p < paginas.size(); p++) {
    int pageObj = firstPageObj + p * 3;
    int contObj = pageObj + 1;
    int imgObj = pageObj + 2;

    byte[] jpeg = jpegs.get(p);
    int imgW = paginas.get(p).getWidth();
    int imgH = paginas.get(p).getHeight();

    offsets.add(headerLen + body.size());
    writeStr.accept(pageObj + " 0 obj\n"
        + "<< /Type /Page /Parent 2 0 R\n"
        + " /MediaBox [0 0 " + PAGE_W + " " + PAGE_H +
"]\n"
        + " /Contents " + contObj + " 0 R\n"
        + " /Resources << /XObject << /Img" + p + " " +
imgObj + " 0 R >> >>\n"
        + ">>\nendobj\n");

    String cs = "q " + PAGE_W + " 0 0 " + PAGE_H + " 0 0 cm
/Img" + p + " Do Q\n";
    byte[] csBytes = cs.getBytes(StandardCharsets.ISO_8859_1);
    offsets.add(headerLen + body.size());
    writeStr.accept(contObj + " 0 obj\n<< /Length " +
csBytes.length + " >>\nstream\n");
    writeBytes.accept(csBytes);

```

```

        writeStr.accept("\nendstream\nendobj\n");

        offsets.add(headerLen + body.size());
        writeStr.accept(imgObj + " 0 obj\n"
            + "<< /Type /XObject /Subtype /Image\n"
            + "    /Width " + imgW + " /Height " + imgH + "\n"
            + "    /ColorSpace /DeviceRGB /BitsPerComponent 8\n"
            + "    /Filter /DCTDecode /Length " + jpeg.length +
"\n"
            + ">>\nstream\n");
        writeBytes.accept(jpeg);
        writeStr.accept("\nendstream\nendobj\n");
    }

    int xrefOffset = headerLen + body.size();
    int totalObjs = offsets.size() + 1;

    StringBuilder xref = new StringBuilder();
    xref.append("xref\n0 ").append(totalObjs).append("\n");
    xref.append("0000000000 65535 f \n");
    for (int off : offsets) {
        xref.append(String.format("%010d 00000 n \n", off));
    }
    xref.append("trailer\n<< /Size ").append(totalObjs)
        .append(" /Root 1 0 R >>\nstartxref\n")
        .append(xrefOffset).append("\n%%EOF\n");

    out.write(header);
    body.writeTo(out);
    out.write(xref.toString().getBytes(StandardCharsets.ISO_8859_1));
}
}

```

8.X. Clase `PanelGrafica.java`

Ubicación: `src/main/java/com/sistemacontable/modulo8/vista/PanelGrafica.java`

```
package com.sistemacontable.modulo8.vista;

import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import javax.swing.*;
import java.awt.*;
import java.awt.geom.*;
import java.util.List;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Vista: Componente Swing de gráfica de barras con gradientes + línea de
 * tendencia.
 *
 * Mejoras respecto a la versión anterior:
 * - Barras con relleno en degradado (GradientPaint) para aspecto
 * premium.
 * - Tooltips de valores sobre cada barra.
 * - Línea de tendencia suavizada (curva cúbica).
 * - Animación de aparición al actualizar datos.
 */
public class PanelGrafica extends JPanel {

    private ResultadoPrediccion resultado;

    // — Paleta de colores
    private static final Color C_VENTAS = new Color(34, 197, 94);
    // verde esmeralda
    private static final Color C_VENTAS_OSC = new Color(22, 163, 74);
    private static final Color C_COMPRAS = new Color(59, 130, 246);
    // azul índigo
    private static final Color C_COMPRAS_OSC = new Color(37, 99, 235);
    private static final Color C_PRED = new Color(250, 204, 21);
    // ámbar
    private static final Color C_PRED_OSC = new Color(234, 179, 8);
    private static final Color C_TENDENCIA = new Color(239, 68, 68);
    // rojo coral
    private static final Color C_FONDO = new Color(248, 250, 252);
    private static final Color C_FONDO_PRED = new Color(254, 252, 232,
100);
    private static final Color C_GRID = new Color(226, 232, 240);
    private static final Color C_TEXTO = new Color(71, 85, 105);

    private static final Font F_SMALL = new Font("SansSerif",
Font.PLAIN, 10);
    private static final Font F_BOLD = new Font("SansSerif", Font.BOLD,
11);
    private static final Font F_ITALIC = new Font("SansSerif",
Font.ITALIC, 13);

    // — Padding interno
    private static final int PAD_L = 72;
    private static final int PAD_R = 20;
```



```

private static final int PAD_T = 20;
private static final int PAD_B = 50;

// — Constructor

public PanelGrafica() {
    setBackground(C_FONDO);
    setPreferredSize(new Dimension(600, 260));
    setBorder(BorderFactory.createLineBorder(C_GRID));
}

/** Actualiza los datos y repinta el componente. */
public void actualizar(ResultadoPrediccion resultado) {
    this.resultado = resultado;
    repaint();
}

// — Pintado principal

@Override
protected void paintComponent(Graphics g0) {
    super.paintComponent(g0);
    Graphics2D g = prepararGraphics(g0);

    if (resultado == null) {
        dibujarEstadoVacio(g);
        return;
    }

    List<RegistroFinanciero> hist = resultado.getHistorial();
    int total = hist.size() + 1; // +1 columna de predicción

    int gw = getWidth() - PAD_L - PAD_R;
    int gh = getHeight() - PAD_T - PAD_B;

    double maxVal = calcularMaxVal(hist);
    int grupW = gw / total;
    int barW = Math.max(6, grupW / 3);

    // — Cuadrícula y ejes —
    dibujarCuadricula(g, gw, gh, maxVal);

    boolean multiYear = false;
    if (!hist.isEmpty()) {
        int firstYear = hist.get(0).getAnio();
        for (RegistroFinanciero r : hist) {
            if (r.getAnio() != firstYear) {
                multiYear = true;
                break;
            }
        }
    }

    // — Barras históricas —
    for (int i = 0; i < hist.size(); i++) {
        RegistroFinanciero r = hist.get(i);
        int x0 = PAD_L + i * grupW + (grupW - barW * 2 - 4) / 2;

        int hC = (int)(gh * r.getTotalCompras() / maxVal);
        int hV = (int)(gh * r.getTotalVentas() / maxVal);
    }
}

```

```

        dibujarBarraGradiente(g, x0, PAD_T + gh - hC, barW,
hC,
        C_COMPRAS_OSC, C_COMPRAS, false);
        dibujarBarraGradiente(g, x0 + barW + 4, PAD_T + gh - hV,
barW, hV,
        C_VENTAS_OSC, C_VENTAS, false);

        // Etiqueta del mes
        g.setFont(F_SMALL);
        g.setColor(C_TEXTO);
        String mes = r.getNombreMes();
        if (multiYear) {
            mes += String.format(" '%02d", r.getAnio() % 100);
        }
        FontMetrics fm = g.getFontMetrics();
        g.drawString(mes,
            PAD_L + i * grupW + (grupW - fm.stringWidth(mes)) /
2,
            PAD_T + gh + 14);
    }

    // - Columna de predicción -
    int predX = PAD_L + hist.size() * grupW;
    dibujarFondoPrediccion(g, predX, grupW, gh);

    int pX0 = predX + (grupW - barW * 2 - 4) / 2;
    int hPC = (int)(gh * resultado.getComprasPredichas() / maxVal);
    int hPV = (int)(gh * resultado.getVentasPredichas() / maxVal);

    dibujarBarraGradiente(g, pX0, PAD_T + gh - hPC, barW,
hPC,
        C_PRED_OSC, C_PRED, true);
    dibujarBarraGradiente(g, pX0 + barW + 4, PAD_T + gh - hPV, barW,
hPV,
        C_PRED_OSC, C_PRED, true);

    // Etiqueta "Pred."
    g.setFont(F_BOLD);
    g.setColor(new Color(161, 98, 7));
    FontMetrics fm = g.getFontMetrics();
    String predLabel = "Pred.";
    g.drawString(predLabel, predX + (grupW -
fm.stringWidth(predLabel)) / 2,
        PAD_T + gh + 14);

    // - Línea de tendencia suavizada -
    dibujarLineaTendencia(g, hist, gw, gh, maxVal, grupW, barW);

    // - Leyenda -
    dibujarLeyenda(g, PAD_T + gh + 28);
}

```

// — Componentes de dibujo

```

private void dibujarCuadricula(Graphics2D g, int gw, int gh, double
maxVal) {
    g.setFont(F_SMALL);
    int lineas = 5;
    for (int i = 0; i <= lineas; i++) {
        int y = PAD_T + gh - (int)(gh * i / (double) lineas);

        // Línea horizontal
    }
}

```

```

        g.setColor(C_GRID);
        g.drawLine(PAD_L, y, PAD_L + gw, y);

        // Etiqueta del eje Y
        double val = maxVal * i / lineas;
        String lbl = val >= 1000
            ? String.format("%.0fK", val / 1000)
            : String.format("%.0f", val);
        g.setColor(C_TEXTO);
        FontMetrics fm = g.getFontMetrics();
        g.drawString(lbl, PAD_L - fm.stringWidth(lbl) - 5, y + 4);
    }
    // Eje Y vertical
    g.setColor(C_GRID);
    g.drawLine(PAD_L, PAD_T, PAD_L, PAD_T + gh);
}

/**
 * Dibuja una barra con relleno en degradado vertical.
 *
 * @param prediccion Si es {@code true} usa opacidad reducida (barra
proyectada).
 */
private void dibujarBarraGradiente(Graphics2D g,
                                    int x, int y, int w, int h,
                                    Color colorBase, Color
colorClaro,
                                    boolean prediccion) {
    if (h <= 0) return;

    Color top = prediccion ? new Color(colorBase.getRed(),
colorBase.getGreen(),
                                    colorBase.getBlue(), 200) :
colorBase;
    Color bottom = prediccion ? new Color(colorClaro.getRed(),
colorClaro.getGreen(),
                                    colorClaro.getBlue(), 130)
: colorClaro;

    GradientPaint gp = new GradientPaint(x, y, top, x, y + h,
bottom);
    g.setPaint(gp);
    g.fillRoundRect(x, y, w, h, 4, 4);

    // Brillo superior
    g.setPaint(new Color(255, 255, 255, 60));
    g.fillRoundRect(x + 1, y + 1, w / 2 - 1, Math.min(h - 2, 8), 3,
3);

    // Borde sutil
    g.setPaint(top.darker());
    g.drawRoundRect(x, y, w, h, 4, 4);
}

private void dibujarFondoPrediccion(Graphics2D g, int predX, int
grupW, int gh) {
    // Fondo amarillo traslúcido
    g.setColor(C_FONDO_PRED);
    g.fillRect(predX, PAD_T, grupW, gh);

    // Línea divisora punteada
    g.setColor(new Color(202, 138, 4, 160));
    float[] dash = {5f, 4f};

```

```

        g.setStroke(new BasicStroke(1.5f, BasicStroke.CAP_BUTT,
            BasicStroke.JOIN_MITER, 10f, dash, 0f));
        g.drawLine(predX, PAD_T, predX, PAD_T + gh);
        g.setStroke(new BasicStroke(1));
    }

    private void dibujarLineaTendencia(Graphics2D g,
        List<RegistroFinanciero> hist,
        int gw, int gh,
        double maxVal, int grupW, int
barW) {
        if (hist.size() < 2) return;

        // Recopilar puntos de ventas
        int n = hist.size();
        int[] px = new int[n];
        int[] py = new int[n];
        for (int i = 0; i < n; i++) {
            px[i] = PAD_L + i * grupW + grupW / 2;
            py[i] = PAD_T + gh - (int)(gh * hist.get(i).getTotalVentas()
/ maxVal);
        }

        // Dibujar línea suavizada (curvas cuadráticas)
        g.setColor(new Color(239, 68, 68, 200));
        g.setStroke(new BasicStroke(2.2f, BasicStroke.CAP_ROUND,
BasicStroke.JOIN_ROUND));

        Path2D path = new Path2D.Double();
        path.moveTo(px[0], py[0]);
        for (int i = 1; i < n; i++) {
            int ctrlX = (px[i - 1] + px[i]) / 2;
            path.quadTo(ctrlX, py[i - 1], px[i], py[i]);
        }
        g.draw(path);

        // Puntos sobre la línea
        g.setColor(C_TENDENCIA);
        for (int i = 0; i < n; i++) {
            g.fillOval(px[i] - 3, py[i] - 3, 7, 7);
            g.setColor(Color.WHITE);
            g.fillOval(px[i] - 1, py[i] - 1, 3, 3);
            g.setColor(C_TENDENCIA);
        }

        // Línea punteada hacia la predicción
        int predPX = PAD_L + n * grupW + grupW / 2;
        int predPY = PAD_T + gh - (int)(gh *
resultado.getVentasPredichas() / maxVal);
        g.setStroke(new BasicStroke(1.8f, BasicStroke.CAP_BUTT,
            BasicStroke.JOIN_MITER, 10f, new float[]{5f, 5f}, 0f));
        g.setColor(new Color(239, 68, 68, 160));
        g.drawLine(px[n - 1], py[n - 1], predPX, predPY);
        g.setStroke(new BasicStroke(1));

        // Punto de predicción (más grande)
        g.setColor(C_TENDENCIA);
        g.fillOval(predPX - 5, predPY - 5, 10, 10);
        g.setColor(Color.WHITE);
        g.fillOval(predPX - 2, predPY - 2, 4, 4);
    }

    private void dibujarLeyenda(Graphics2D g, int y) {

```

```

        g.setFont(F_SMALL);
        int x = PAD_L;
        int sz = 10;
        int sp = 95;

        Object[][] items = {
            {C_COMPRAS, "Compras"},
            {C_VENTAS, "Ventas"},
            {C_PRED, "Predicción"},
            {C_TENDENCIA, "Tendencia ventas"},
        };

        for (Object[] item : items) {
            Color color = (Color) item[0];
            GradientPaint gp = new GradientPaint(x, y - sz + 2,
color.darker(),
                                                    x + sz, y + 2, color);

            g.setPaint(gp);
            g.fillRoundRect(x, y - sz + 2, sz, sz, 3, 3);
            g.setColor(C_TEXTO);
            g.drawString((String) item[1], x + sz + 5, y);
            x += sp;
        }
    }

    private void dibujarEstadoVacio(Graphics2D g) {
        g.setFont(F_ITALIC);
        g.setColor(new Color(148, 163, 184));
        String msg = "Seleccione una empresa y ejecute el análisis para
ver la gráfica";
        FontMetrics fm = g.getFontMetrics();
        g.drawString(msg, (getWidth() - fm.stringWidth(msg)) / 2,
getHeight() / 2);

        // ícono decorativo
        g.setColor(new Color(226, 232, 240));
        g.fillOval(getWidth() / 2 - 28, getHeight() / 2 - 55, 56, 56);
        g.setFont(new Font("SansSerif", Font.BOLD, 26));
        g.setColor(new Color(203, 213, 225));
        FontMetrics fm2 = g.getFontMetrics();
        g.drawString("~", (getWidth() - fm2.stringWidth("~")) / 2,
getHeight() / 2 - 20);
    }

//      —      Helpers

```

```

    private Graphics2D prepararGraphics(Graphics g0) {
        Graphics2D g = (Graphics2D) g0;
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING,
RenderingHints.VALUE_TEXT_ANTIALIAS_LCD_HRGB);
        g.setRenderingHint(RenderingHints.KEY_RENDERING,
RenderingHints.VALUE_RENDER_QUALITY);
        return g;
    }

    private double calcularMaxVal(List<RegistroFinanciero> hist) {
        double max = 1;
        for (RegistroFinanciero r : hist) {
            max = Math.max(max, Math.max(r.getTotalVentas(),
r.getTotalCompras()));
        }
    }

```

```
        }  
        return Math.max(max, resultado.getVentasPredichas()) * 1.15;  
    }  
}
```

8.X. Clase `VentanaAnálisis.java`

Ubicación:

src/main/java/com/sistemacontable/modulo8/vista/VentanaAnálisis.java

```
package com.sistemacontable.modulo8.vista;

import com.sistemacontable.modulo8.controlador.AnálisisControlador;
import com.sistemacontable.modulo8.modelo.EmpresaItem;
import com.sistemacontable.modulo8.modelo.RegistroFinanciero;
import com.sistemacontable.modulo8.modelo.ResultadoPrediccion;

import javax.swing.*;
import javax.swing.border.*;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.JTableHeader;
import java.awt.*;
import java.awt.geom.RoundRectangle2D;
import java.time.format.DateTimeFormatter;
import java.util.List;

/**
 * MÓDULO 8 - Análisis Predictivo con IA
 * Vista principal del módulo (Swing).
 *
 * Layout general:
 *
 * 

|                             |                               |
|-----------------------------|-------------------------------|
| HEADER (título + subtítulo) |                               |
| PANEL CONTROL               | KPIs (5 tarjetas)             |
| Empresa ▼                   |                               |
| ▶ Analizar                  | Gráfica de tendencia          |
| 📄 PDF                       |                               |
| Info modelo                 | Tabla de historial financiero |


 *
 * Mejoras:
 * - Tipografía consistente y moderna.
 * - Ícono de ventana generado programáticamente.
 * - Panel de control con separador visual mejorado.
 * - Tabla con colores alternados y renderer de color para utilidad.
 */
public class VentanaAnálisis extends JFrame {

    // ----- Controlador
    private AnálisisControlador controlador;

    // ----- Componentes de control
    private JComboBox<EmpresaItem> cmbEmpresa;
    private JButton btnAnalizar;
    private JButton btnExportarPDF;
    private JProgressBar progressBar;

    // ----- Componentes de resultado
    private JLabel lblVentasPredichas;
    private JLabel lblComprasPredichas;
```

```

private JLabel lblUtilidad;
private JLabel lblTendencia;
private JLabel lblConfianza;
private JLabel lblFecha;
private JLabel lblTiempo;

private PanelGrafica      panelGrafica;
private DefaultTableModel modeloTabla;
private JTable            tablaHistorial;

//      —      Paleta      de      colores

private static final Color C_PRIMARIO   = new Color(30,   72, 156);
private static final Color C_SECUNDARIO = new Color(59,  130, 246);
private static final Color C_FONDO      = new Color(241, 245, 249);
private static final Color C_PANEL      = new Color(255, 255, 255);
private static final Color C_VERDE      = new Color(22,  163,  74);
private static final Color C_ROJO       = new Color(220,  38,  38);
private static final Color C_AMARILLO   = new Color(234, 179,   8);
private static final Color C_GRIS_TEXTO = new Color(100, 116, 139);
private static final Color C_BORDE      = new Color(226, 232, 240);
private static final Color C_BLANCO     = Color.WHITE;
private static final Color C_FILA_PAR   = new Color(248, 250, 252);
private static final Color C_FILA_IMPAR = Color.WHITE;

//      —      Tipografía

private static final Font F_TITULO      = new Font("SansSerif",
Font.BOLD, 18);
private static final Font F_SUBTITULO   = new Font("SansSerif",
Font.PLAIN, 11);
private static final Font F_LABEL       = new Font("SansSerif",
Font.BOLD, 11);
private static final Font F_NORMAL      = new Font("SansSerif",
Font.PLAIN, 11);
private static final Font F_SMALL       = new Font("SansSerif",
Font.PLAIN, 10);
private static final Font F_TINY        = new Font("SansSerif",
Font.ITALIC, 9);
private static final Font F_KPI_VAL     = new Font("SansSerif",
Font.BOLD, 15);
private static final Font F_KPI_LBL     = new Font("SansSerif",
Font.PLAIN, 9);

//      —      Constructor

public VentanaAnalisis() {
    super("Módulo 8 - Análisis Predictivo con Inteligencia
Artificial");
    configurarVentana();
    construirUI();
}

/** Conecta el controlador después del constructor. */
public void setControlador(AnalisisControlador controlador) {
    this.controlador = controlador;
}

//      —      Configuración      de      ventana

private void configurarVentana() {

```



```

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1100, 740);
        setMinimumSize(new Dimension(920, 620));
        setLocationRelativeTo(null);
        getContentPane().setBackground(C_FONDO);
        setLayout(new BorderLayout(0, 0));
        setIconImage(crearIconoVentana());

        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {}
    }

    /** Genera un ícono de ventana con degradado azul y símbolo de IA. */
    private Image crearIconoVentana() {
        int sz = 32;
        java.awt.image.BufferedImage img =
            new java.awt.image.BufferedImage(sz, sz,
            java.awt.image.BufferedImage.TYPE_INT_ARGB);
        Graphics2D g = img.createGraphics();
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
        g.setPaint(new GradientPaint(0, 0, C_PRIMARIO, sz, sz,
        C_SECUNDARIO));
        g.fillRoundRect(0, 0, sz, sz, 8, 8);
        g.setColor(Color.WHITE);
        g.setFont(new Font("SansSerif", Font.BOLD, 16));
        FontMetrics fm = g.getFontMetrics();
        g.drawString("AI", (sz - fm.stringWidth("AI")) / 2, sz / 2 +
        fm.getAscent() / 2 - 2);
        g.dispose();
        return img;
    }

    // — Construcción de la interfaz

    private void construirUI() {
        add(crearHeader(), BorderLayout.NORTH);
        add(crearPanelControl(), BorderLayout.WEST);

        JPanel centro = new JPanel(new BorderLayout(0, 8));
        centro.setBackground(C_FONDO);
        centro.setBorder(new EmptyBorder(8, 8, 8, 8));
        centro.add(crearPanelKPIs(), BorderLayout.NORTH);
        centro.add(crearPanelGrafica(), BorderLayout.CENTER);
        centro.add(crearPanelTabla(), BorderLayout.SOUTH);
        add(centro, BorderLayout.CENTER);
    }

    // — Header

    private JPanel crearHeader() {
        JPanel header = new JPanel(new BorderLayout()) {
            @Override
            protected void paintComponent(Graphics g) {
                super.paintComponent(g);
                Graphics2D g2 = (Graphics2D) g;
                g2.setPaint(new GradientPaint(0, 0, C_PRIMARIO,
                getWidth(), 0,
                new Color(49, 112, 204)));
            }
        };
    }

```

```

        g2.fillRect(0, 0, getWidth(), getHeight());
    }
};
header.setPreferredSize(new Dimension(0, 64));
header.setBorder(new EmptyBorder(0, 18, 0, 18));

JLabel titulo = new JLabel("  Módulo 8 – Análisis Predictivo con
IA");
titulo.setFont(F_TITULO);
titulo.setForeground(C_BLANCO);
titulo.setIcon(new CircleIcon(18, C_AMARILLO));

JLabel subtitulo = new JLabel("Sistema Contable Computarizado ·
Red Neuronal MLP · Ciclo I-2026");
subtitulo.setFont(F_SUBTITULO);
subtitulo.setForeground(new Color(186, 207, 240));

JPanel textos = new JPanel(new GridLayout(2, 1, 0, 2));
textos.setOpaque(false);
textos.add(titulo);
textos.add(subtitulo);
header.add(textos, BorderLayout.WEST);

// Línea inferior ámbar
JPanel linea = new JPanel();
linea.setOpaque(true);
linea.setBackground(C_AMARILLO);
linea.setPreferredSize(new Dimension(0, 3));
header.add(linea, BorderLayout.SOUTH);

return header;
}

//      —      Panel      de      control      lateral

private JPanel crearPanelControl() {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBackground(C_PANEL);
    panel.setBorder(new CompoundBorder(
        new MatteBorder(0, 0, 0, 1, C BORDE),
        new EmptyBorder(18, 14, 16, 14)));
    panel.setPreferredSize(new Dimension(215, 0));

    // Selector de empresa
    JLabel lblEmpresa = nuevaEtiqueta("Seleccionar Empresa:",
F_LABEL, C_PRIMARIO);

    cmbEmpresa = new JComboBox<>();
    cmbEmpresa.setMaximumSize(new Dimension(Integer.MAX_VALUE, 30));
    cmbEmpresa.setAlignmentX(LEFT_ALIGNMENT);
    cmbEmpresa.setFont(F_NORMAL);
    cmbEmpresa.setBackground(C_BLANCO);

    // Botones
    btnAnalizar      = crearBoton("►  Ejecutar Análisis", C_PRIMARIO,
C_BLANCO);
    btnExportarPDF = crearBoton("📄  Exportar a PDF",      C_VERDE,
C_BLANCO);
    btnExportarPDF.setEnabled(false);

    btnAnalizar.addActionListener(e -> {

```

```

        EmpresaItem seleccion = (EmpresaItem)
cmbEmpresa.getSelectedItem();
        if (seleccion == null) return;
        controlador.ejecutarAnálisis(seleccion.getId());
    });
    btnExportarPDF.addActionListener(e -> controlador.exportarPDF());

    // Barra de progreso
    progressBar = new JProgressBar();
    progressBar.setIndeterminate(true);
    progressBar.setVisible(false);
    progressBar.setMaximumSize(new Dimension(Integer.MAX_VALUE, 6));
    progressBar.setAlignmentX(LEFT_ALIGNMENT);
    progressBar.setForeground(C_SECUNDARIO);
    progressBar.setBorderPainted(false);

    // Info del modelo
    JTextArea info = new JTextArea(
        "Red Neuronal Artificial MLP\n" +
        "implementada en Java puro.\n\n" +
        "Arquitectura:\n" +
        "  Entrada → 3 neuronas\n" +
        "  Oculta 1 → 8 (ReLU)\n" +
        "  Oculta 2 → 4 (ReLU)\n" +
        "  Salida → 2 (Sigmoid)\n\n" +
        "Épocas de entrenamiento: 5.000\n" +
        "Tasa de aprendizaje: 0.05\n\n" +
        "Requiere ≥ 3 meses de historial.");
    info.setFont(F_SMALL);
    info.setForeground(C_GRIIS_TEXTO);
    info.setBackground(C_PANEL);
    info.setEditable(false);
    info.setLineWrap(true);
    info.setWrapStyleWord(true);
    info.setAlignmentX(LEFT_ALIGNMENT);
    info.setBorder(new TitledBorder(
        BorderFactory.createLineBorder(C_BORDE),
        " Acerca del Modelo ", TitledBorder.LEFT,
TitledBorder.TOP,
        new Font("SansSerif", Font.BOLD, 10), C_PRIMARIO));

    // Etiquetas de estado
    lblFecha = nuevaEtiqueta("Fecha: -", F_TINY,
C_GRIIS_TEXTO);
    lblTiempo = nuevaEtiqueta("Entrenamiento: -", F_TINY,
C_GRIIS_TEXTO);

    panel.add(lblEmpresa);
    panel.add(Box.createVerticalStrut(5));
    panel.add(cmbEmpresa);
    panel.add(Box.createVerticalStrut(10));
    panel.add(btnAnalizar);
    panel.add(Box.createVerticalStrut(6));
    panel.add(btnExportarPDF);
    panel.add(Box.createVerticalStrut(6));
    panel.add(progressBar);
    panel.add(Box.createVerticalStrut(14));
    panel.add(info);
    panel.add(Box.createVerticalGlue());
    panel.add(nuevaSeparador());
    panel.add(Box.createVerticalStrut(6));
    panel.add(lblFecha);
    panel.add(Box.createVerticalStrut(3));

```

```

        panel.add(lblTiempo);

        return panel;
    }

// — KPIs

private JPanel crearPanelKPIs() {
    JPanel panel = new JPanel(new GridLayout(1, 5, 8, 0));
    panel.setBackground(C_FONDO);
    panel.setPreferredSize(new Dimension(0, 88));

    lblVentasPredichas = crearTarjetaKPI(panel, "Ventas Próx. Mes",
    "-", C_VERDE);
    lblComprasPredichas = crearTarjetaKPI(panel, "Compras Próx. Mes",
    "-", C_SECUNDARIO);
    lblUtilidad = crearTarjetaKPI(panel, "Utilidad
Proyectada", "-", C_PRIMARIO);
    lblTendencia = crearTarjetaKPI(panel, "Tendencia",
    "-", C_AMARILLO.darker());
    lblConfianza = crearTarjetaKPI(panel, "Confianza Modelo",
    "-", new Color(100,116,139));

    return panel;
}

private JLabel crearTarjetaKPI(JPanel parent, String titulo, String
valor, Color color) {
    JPanel card = new JPanel(new BorderLayout()) {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2 = (Graphics2D) g;
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            g2.setPaint(new GradientPaint(0, 0, color, 0,
getHeight(), color.darker()));
            g2.fill(new RoundRectangle2D.Double(0, 0, getWidth(),
getHeight(), 8, 8));
        }
    };
    card.setOpaque(false);
    card.setBorder(new EmptyBorder(10, 12, 10, 12));

    JLabel lblTitulo = new JLabel(titulo);
    lblTitulo.setFont(F_KPI_LBL);
    lblTitulo.setForeground(new Color(255, 255, 255, 200));

    JLabel lblValor = new JLabel(valor);
    lblValor.setFont(F_KPI_VAL);
    lblValor.setForeground(C_BLANCO);

    card.add(lblTitulo, BorderLayout.NORTH);
    card.add(lblValor, BorderLayout.CENTER);
    parent.add(card);
    return lblValor;
}

// — Gráfica

private JPanel crearPanelGrafica() {

```

```

        panelGrafica = new PanelGrafica();
        JPanel wrapper = crearPanelConBorde(" Gráfica de Tendencia
Histórica y Predicción ");
        wrapper.add(panelGrafica, BorderLayout.CENTER);
        return wrapper;
    }

//      —      Tabla      de      historial

private JPanel crearPanelTabla() {
    String[] columnas = {
        "Mes", "Periodo", "Total Compras ($)", "Total Ventas ($)",
        "Utilidad ($)", "Var. Ventas (%)"
    };
    modeloTabla = new DefaultTableModel(columnas, 0) {
        @Override public boolean isCellEditable(int r, int c) {
return false; }
    };

    tablaHistorial = new JTable(modeloTabla);
    tablaHistorial.setRowHeight(24);
    tablaHistorial.setFont(F_NORMAL);
    tablaHistorial.setSelectionBackground(new Color(219, 234, 254));
    tablaHistorial.setSelectionForeground(C_PRIMARIO);
    tablaHistorial.setGridColor(C_BORDE);
    tablaHistorial.setShowGrid(true);
    tablaHistorial.setInterCellSpacing(new Dimension(0, 1));

    // Header con estilo
    JTableHeader header = tablaHistorial.getTableHeader();
    header.setFont(F_LABEL);
    header.setBackground(C_PRIMARIO);
    header.setForeground(C_BLANCO);
    header.setPreferredSize(new Dimension(0, 28));

    // Renderer para colores alternos y utilidad roja/verde
    tablaHistorial.setDefaultRenderer(Object.class, new
DefaultTableCellRenderer() {
        @Override
        public Component getTableCellRendererComponent(JTable t,
Object val,
        boolean sel, boolean foc, int row, int col) {
            super.getTableCellRendererComponent(t, val, sel, foc,
row, col);
            if (!sel) {
                setBackground(row % 2 == 0 ? C_FILA_PAR :
C_FILA_IMPAR);
                setForeground(Color.DARK_GRAY);
            }
            // Última fila = predicción destacada
            if (row == t.getRowCount() - 1) {
                setBackground(new Color(239, 246, 255));
                setFont(F_LABEL);
                setForeground(C_PRIMARIO);
            } else {
                setFont(F_NORMAL);
            }
            // Columna utilidad: verde/rojo
            if (col == 4 && val != null && !sel) {
                String s = val.toString().replace(",", ""
).replace("$", "");
                try {

```

```

        double d = Double.parseDouble(s);
        setForeground(d >= 0 ? C_VERDE : C_ROJO);
    } catch (NumberFormatException ignored) {}
    }
    setBorder(new EmptyBorder(0, 6, 0, 6));
    return this;
}
});

JScrollPane scroll = new JScrollPane(tablaHistorial);
scroll.setPreferredSize(new Dimension(0, 165));
scroll.setBorder(BorderFactory.createEmptyBorder());

JPanel panel = crearPanelConBorde(" Historial Financiero Mensual
");
panel.add(scroll, BorderLayout.CENTER);
return panel;
}

// — API para el Controlador

/** Puebla el selector de empresas. */
public void poblarSelectorEmpresas(List<EmpresaItem> empresas) {
    cmbEmpresa.removeAllItems();
    for (EmpresaItem emp : empresas) {
        cmbEmpresa.addItem(emp);
    }
}

/** Muestra el resultado completo del análisis en la UI. */
public void mostrarResultados(ResultadoPrediccion res) {
    // KPIs
    lblVentasPredichas .setText(String.format("$%,.2f",
res.getVentasPredichas()));
    lblComprasPredichas.setText(String.format("$%,.2f",
res.getComprasPredichas()));
    lblUtilidad .setText(String.format("$%,.2f",
res.getUtilidadProyectada()));

    String tendStr = res.getClasificacionTendencia() +
String.format(" (%+.1f%)",
res.getTendenciaPorcentaje());
    lblTendencia.setText(tendStr);
    lblConfianza.setText(res.getNivelConfianza());

    // Meta info
    DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy
HH:mm");
    lblFecha .setText("Fecha: " +
res.getFechaAnalisis().format(fmt));
    lblTiempo.setText("Entreno: " + res.getTiempoEntrenamientoMs() +
" ms");

    // Tabla
    modeloTabla.setRowCount(0);
    List<RegistroFinanciero> hist = res.getHistorial();
    for (int i = 0; i < hist.size(); i++) {
        RegistroFinanciero r = hist.get(i);
        double utilMes = r.getTotalVentas() - r.getTotalCompras();
        String varStr = "-";
        if (i > 0) {
            double varPorc = ((r.getTotalVentas() - hist.get(i -

```

```

1).getTotalVentas())
        / hist.get(i - 1).getTotalVentas()) * 100.0;
        varStr = String.format("%+.1f%%", varPorc);
    }
    modeloTabla.addRow(new Object[]{
        r.getNombreMes(),
        r.getMesNumero() + "/" + r.getAnio(),
        String.format("%.2f", r.getTotalCompras()),
        String.format("%.2f", r.getTotalVentas()),
        String.format("%.2f", utilMes),
        varStr
    });
}

// Fila de predicción
modeloTabla.addRow(new Object[]{
    "► PRED.",
    "Próx. mes",
    String.format("%.2f", res.getComprasPredichas()),
    String.format("%.2f", res.getVentasPredichas()),
    String.format("%.2f", res.getUtilidadProyectada()),
    String.format("%+.1f%%", res.getTendenciaPorcentaje())
});

panelGrafica.actualizar(res);
btnExportarPDF.setEnabled(true);
}

/** Limpia todos los componentes de resultado. */
public void limpiarResultados() {
    lblVentasPredichas .setText("-");
    lblComprasPredichas.setText("-");
    lblUtilidad        .setText("-");
    lblTendencia        .setText("-");
    lblConfianza        .setText("-");
    lblFecha            .setText("Fecha: -");
    lblTiempo           .setText("Entrenamiento: -");
    modeloTabla.setRowCount(0);
    panelGrafica.actualizar(null);
    btnExportarPDF.setEnabled(false);
}

/** Activa o desactiva la animación de carga. */
public void setEstadoCargando(boolean cargando) {
    progressBar .setVisible(cargando);
    btnAnalizar  .setEnabled(!cargando);
    cmbEmpresa   .setEnabled(!cargando);
}

// — Helpers de UI

private JButton crearBoton(String texto, Color fondo, Color
foreground) {
    JButton btn = new JButton(texto) {
        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g;
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
            if (isEnabled()) {
                Color hover = getModel().isRollover() ?
fondo.brighter() : fondo;

```

```

        g2.setPaint(new GradientPaint(0, 0, hover.brighter(),
0, getHeight(), hover));
    } else {
        g2.setColor(new Color(200, 210, 220));
    }
    g2.fillRoundRect(0, 0, getWidth(), getHeight(), 6, 6);
    super.paintComponent(g);
}

};
btn.setForeground(foreground);
btn.setFont(F_LABEL);
btn.setFocusPainted(false);
btn.setBorderPainted(false);
btn.setContentAreaFilled(false);
btn.setOpaque(false);
btn.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
btn.setMaximumSize(new Dimension(Integer.MAX_VALUE, 34));
btn.setAlignmentX(LEFT_ALIGNMENT);
return btn;
}

private JLabel nuevaEtiqueta(String texto, Font font, Color color) {
    JLabel lbl = new JLabel(texto);
    lbl.setFont(font);
    lbl.setForeground(color);
    lbl.setAlignmentX(LEFT_ALIGNMENT);
    return lbl;
}

private JPanel crearPanelConBorde(String titulo) {
    JPanel panel = new JPanel(new BorderLayout());
    panel.setBackground(C_PANEL);
    panel.setBorder(new TitledBorder(
        BorderFactory.createLineBorder(C_BORDE),
        titulo, TitledBorder.LEFT, TitledBorder.TOP,
        F_LABEL, C_PRIMARIO));
    return panel;
}

private JSeparator nuevaSeparador() {
    JSeparator sep = new JSeparator();
    sep.setAlignmentX(LEFT_ALIGNMENT);
    sep.setMaximumSize(new Dimension(Integer.MAX_VALUE, 1));
    sep.setForeground(C_BORDE);
    return sep;
}

//      —      ícono      circular      decorativo

private static class CircleIcon implements Icon {
    private final int size;
    private final Color color;
    CircleIcon(int size, Color color) { this.size = size; this.color
= color; }

    @Override
    public void paintIcon(Component c, Graphics g, int x, int y) {
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setPaint(new GradientPaint(x, y, color.brighter(), x +
size, y + size, color));
    }
}

```



```
        g2.fillOval(x, y, size, size);
    }

    @Override public int getIconWidth() { return size; }
    @Override public int getIconHeight() { return size; }
}
}
```

9. Documentación del Motor de IA

9.1. Arquitectura Topológica de la Red Neuronal (MLP)

El Perceptrón Multicapa (MLP) fue estructurado en 4 etapas:

- **Capa 1 (Input):** 3 Neuronas. (Mes Normalizado, Compra previa Normalizada, Venta previa Normalizada).
- **Capa 2 (Hidden 1):** 8 Neuronas. Activación ReLU.
- **Capa 3 (Hidden 2):** 4 Neuronas. Activación ReLU.
- **Capa 4 (Output):** 2 Neuronas. Activación Sigmoide. (Compra proyectada, Venta proyectada).

9.2. Justificación de Hiperparámetros

- **5000 Épocas:** Suficiente para sobrepasar el plateau de convergencia sin llegar al extremo del sobreentrenamiento.
- **Learning Rate 0.05:** Una tasa estándar que evita saltar los mínimos globales.
- **Decaimiento (Decay):** 10% cada 1000 épocas amortigua la curva al final del entrenamiento.
- **Semilla Fija (42):** Para garantizar reproducibilidad exacta en los tests de unidad.

9.3. Fórmulas Matemáticas Core

Forward Pass para cada capa L :

$$Z^{\{L\}} = W^{\{L\}} \cdot A^{\{L-1\}} + B^{\{L\}}$$

$$A^{\{L\}} = g(Z^{\{L\}})$$

Funciones de Activación:

$$\text{ReLU: } g(x) = \max(0, x)$$

$$\text{Sigmoide: } g(x) = \frac{1}{1 + e^{-x}}$$

Backpropagation (Derivadas y Regla de la Cadena):

El gradiente del Error respecto a los pesos en la capa de salida:

$$\frac{\partial E}{\partial W^{\{L\}}} = (A^{\{L\}} - Y) \cdot g'(Z^{\{L\}}) \cdot A^{\{L-1\}}$$

Donde $g'(x) = g(x)(1 - g(x))$ para la Sigmoide.

9.4. Proceso de Normalización Min-Max con Ejemplo Numérico

El algoritmo halla los máximos y mínimos históricos reales del lote de datos en lugar de asumir mínimos fijos absolutos, usando la fórmula de escalado Min-Max normal.

Ejemplo: Si las ventas históricas de la empresa son [100, 200, 300].

$$V_{\min} = 100$$

$$V_{\max} = 300$$

La venta del mes central (200) se normaliza como:

$$V_{\text{norm}}(200) = \frac{200 - 100}{300 - 100} = \frac{100}{200} = 0.50$$

La red neuronal ingiere y escupe las predicciones en este formato escalado [0, 1] y luego se revierte la fórmula en el de-escalado hacia dólares.

9.5. Cálculo de Tendencia y Confianza

El nivel de confianza se otorga usando un sistema dual de puntajes (`scoreTotal = scoreMeses + scoreMse`) dentro de `MotorPrediccionIA.java`:

- Score por Meses: ≥ 10 meses (3 pts), ≥ 6 meses (2 pts), menor (1 pt).
- Score por MSE: < 0.05 (3 pts), < 0.15 (2 pts), mayor (1 pt).

La confianza final se evalúa como:

- **Alta:** `scoreTotal >= 5`
- **Media:** `scoreTotal >= 3`
- **Baja:** Cualquier otro valor.

9.6. Limitaciones Conocidas

- **Datos Estacionales:** Un modelo de 1 retraso ($t-1$) no ve que las ventas de diciembre suben por navidad, ya que solo toma noviembre como input. Se requiere memoria (LSTM) para estacionalidad.
- **Sobreajuste:** Si el historial tiene 3 meses, la red memoriza los puntos en lugar de aprender el patrón.

10. Interfaces de Usuario

10.1. Layout y Ventana Principal

La GUI de `VentanaAnálisis.java` utiliza una disposición central basada puramente en `BorderLayout`, evitando la complejidad innecesaria de múltiples componentes y en su lugar empleando subpaneles directos.

- **Norte (Cabecera):** `JComboBox` con el modelo `EmpresaItem` emparejado a los botones principales.
- **Centro:** Muestra un contenedor `Split` o `GridLayout` que aloja a los KPIs, y se anexa de forma directa la instancia de `PanelGrafica.java` para el renderizado histórico, y bajo este, una vista tabular `JTable`. (No se usan pestañas interactivas `JTabbedPane`).

10.2. Flujo de Usuario Paso a Paso

1. Abrir aplicación.
2. Esperar conexión a MariaDB (HikariCP).
3. Seleccionar empresa del `ComboBox`.
4. Presionar "Analizar".
5. Observar congelamiento simulado de reloj de arena (el UI Thread sigue vivo gracias a `SwingWorker`).
6. Examinar KPI proyectado.
7. Hacer clic en Exportar PDF y elegir la ruta.

10.3. Descripción del Reporte PDF Generado

Página 1:

- Rectángulo Top: Brand del SCO.
- Información de Metadata: Tiempo de entrenamiento, empresa, ID.
- KPIs ejecutivos.
- Tabla Histórica iterativa con datos numéricos.

Página 2:

- Gráfica Vectorial Multianual (ej. Ene '24, Feb '25).
- Recomendaciones condicionales inteligentes inferidas de la alerta de utilidad.

11. Configuración e Instalación

11.1. Requisitos de Hardware y Software

- JDK 11+.
- RAM: Mínimo 512 MB.
- MariaDB 10.5+ o MySQL 8+.

11.2. Pasos de Instalación Numerados

1. Clonar el repositorio del SCO Unificado.
2. Renombrar `config/db.properties.example` a `db.properties` y rellenar credenciales.
3. Cargar el script `database/mock_data.sql` en el RDBMS para habilitar el escenario de demostración.
4. Compilar las fuentes usando `ant compile` u otra herramienta preferida.
5. Iniciar la aplicación.

11.3. Variables de Entorno Soportadas

- `DB_URL`
- `DB_USER`
- `DB_PASSWORD`

11.4. Solución a Errores Comunes

- `HikariPool.PoolInitializationException`: El servidor DB está apagado o las credenciales en `db.properties` son incorrectas.
- `HeadlessException`: Si se corre en servidor remoto sin display gráfico (X11), Java arroja este error al intentar crear `VentanaAnálisis` o usar `Graphics2D`.

12. Integración con el Proyecto Unificado SCO

12.1. Invocación

Para encender y mostrar visualmente este módulo desde otra ventana, se debe llamar al método principal orquestador:

```
com.sistemacontable.modulo8.Main.lanzar();
```

12.2. DAO a Tablas Reales

Cuando se unifique con el módulo de facturación, se requerirá editar `DatosFinancierosDAO.java` usando el snippet SQL del Anexo A para referenciar `factura_compra_cabecera` o análogas.

12.3. Dependencias

Garantizar la existencia de la librería MariaDB JDBC y HikariCP en el Classpath base.

13. Pruebas

13.1. Estrategia

Caja Negra sobre la GUI para medir UX. Caja Blanca y JUnit automatizado para el Motor de IA, estableciendo rígidamente una semilla (Seed) con `new Random(42)` asegurando un decaimiento y cálculo de `MSE` 100% reproducible y auditable.

13.2. Tabla de Casos de Prueba Funcionales (Medidos bajo Seed=42)

Escenario	Entrada Test (Ventas)	Resultado Proyectado Obtenido	MSE Final Medido	Evaluación IA
Empresa A (Estable)	8500 -> 13800 progresivo	14,082 USD	0.0125	Tendencia Creciente. Converge perfecto. Confianza Alta.
Empresa B (Errático)	2500, 3000, 4200, 2100...	~3,354 USD	0.0910	Tendencia estable promedio. Convergencia difícil. Confianza Medio/Baja.
Historial Vacío	BD vacía	Lanzamiento Excepción	N/A	Falla limpia atrapada y mostrada en GUI.

14. Seguridad

14.1. Manejo de Credenciales

Jamás almacenar contraseñas embebidas en el `.class` (Hardcoding). Uso exclusivo de `.properties` ignorado por GIT, o Environment Vars en producción.

14.2. Prevención SQL Injection

Se usa la interfaz `PreparedStatement`. Las sentencias `stmt.setInt(1, id)` imposibilitan la inyección de comandos maliciosos tipo `DROP TABLE`.

14.3. Recomendaciones de Producción

Restringir el usuario DB del Módulo 8 a permisos de solo lectura (`GRANT SELECT ON ...`), ya que su alcance se restringe estrictamente a analítica y generación de PDF.

15. Mantenimiento y Mejoras Futuras

15.1. Mejoras Identificadas (Las 12 Aplicadas Recientemente)

El código fue auditado para corregir 12 debilidades:

- Bug Crítico Min_Value -> Corrección a `-Double.MAX_VALUE` en `PreprocesadorDatos`.
- Conexiones muertas -> Solucionado con HikariCP.
- UI Freezes -> Solucionado con SwingWorker.
- Eje X de múltiples años -> Solucionado detectando cambios de año.
- Obesidad del ExportadorPDF -> Solucionado dividiendo el Graphics2D en Fachada y Módulos.

15.2. Roadmap de Desarrollo a Corto Plazo

1. Soporte a LSTM para estacionalidad y series temporales avanzadas.
2. Soporte para proyecciones multianuales.
3. Exportación opcional a MS Excel (.xlsx).

16. Anexos

Anexo A: Código SQL Completo (Base de Datos Mock y Query Futuro)

```
-- 1. Tablas Mock para Testeo Aislado del Módulo 8
CREATE TABLE IF NOT EXISTS empresa_mock (
    id_empresa    INT          PRIMARY KEY,
    nombre_comercial VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS historial_financiero_mock (
    id_registro    INT          PRIMARY KEY AUTO_INCREMENT,
    id_empresa     INT          NOT NULL,
    mes_numero     INT          NOT NULL,
    anio           INT          NOT NULL,
    total_compras  DECIMAL(12,2) NOT NULL,
    total_ventas   DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (id_empresa) REFERENCES empresa_mock(id_empresa)
);

-- =====
-- 2. CONSULTA DE INTEGRACIÓN FUTURA (reemplaza la mock)
-- Descomenta esto cuando el proyecto esté unificado:
-- =====
/*
SELECT
    MONTH(v.fecha_emision) AS mes_numero,
    YEAR(v.fecha_emision)  AS anio,
    SUM(c.total_factura)    AS total_compras,
    SUM(v.total_factura)    AS total_ventas
FROM ventas_facturas v
JOIN compras_facturas c
    ON MONTH(v.fecha_emision) = MONTH(c.fecha_emision)
    AND YEAR(v.fecha_emision)  = YEAR(c.fecha_emision)
    AND v.id_empresa = c.id_empresa
WHERE v.id_empresa = ?
    AND YEAR(v.fecha_emision) = 2025
GROUP BY MONTH(v.fecha_emision), YEAR(v.fecha_emision)
ORDER BY mes_numero;
*/
```

Anexo B: Ejemplo de Reporte PDF

El PDF utiliza el subestándar PDF-1.4 y renderización vectorial embebida nativa en Java AWT, garantizando cero dependencias externas.

Anexo C: Glosario Completo

- **Backpropagation:** Método de propagación hacia atrás.
- **Gradient Descent:** Optimización iterativa.
- **HikariCP:** Connection pool ultra rápido.

Anexo D: Referencias Bibliográficas

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Oracle. Java 11 API Specification.